

LLM-based Code Generation for Python Computer Graphics Frameworks from Natural Language Prompts

BSc Thesis Report

Elisavet Kourkounaki

Student ID: CSD5116

csd5116@csd.uoc.gr

Supervisors

Prof. Georgios Papagiannakis,

Dr. Manos Kamarianakis



Department of Computer Science
University of Crete

June 2026

Contents

Acknowledgements	2
Abstract	4
1 Introduction	5
1.1 Motivation	5
1.2 Research Question	5
1.3 Contributions	6
1.4 Scope and Limitations	7
1.5 Structure of this Thesis	7
2 Background	8
2.1 Large Language Models	8
2.2 Computer Graphics Fundamentals	8
2.3 Entity-Component-System Architecture	9
2.4 The Elements Framework	10
2.5 OpenGL and Shader Programming	11
3 Literature Review	13
3.1 Early Text-to-3D Work	13
3.2 Neural 3D Generation Systems	13
3.3 Language-Based Scene Editing	13
3.4 Code Generation for Graphics	14
3.5 Prompt Engineering and Structured Output	14
3.6 Explainable AI in Education	15
3.7 Comparative Summary	16
4 Methodology	18
4.1 Target Framework: Elements	18
4.2 System Design Decisions	18

4.3	System Pipeline	19
4.4	Intermediate Representation (IR) Schema	20
4.5	Prompt Engineering and LLM Integration	22
4.6	Action Cache Mechanism	23
4.7	Runtime Bridge Architecture	25
4.8	Preview / Apply / Undo Workflow	26
4.9	Python Code Generation Pipeline	28
4.10	OBJ and USD Asset Import	29
4.11	3D Model Loading and Texture Rendering	31
4.12	Geometry Factory	32
4.13	Prefab System	33
4.14	Dynamic Object Motion	34
4.14.1	Orbit	35
4.15	Composite Object Generation	35
4.16	Module Organization	37
5	Implementation Details	38
5.1	Shader System	38
5.2	Texture System	39
5.3	Lighting Model	40
5.4	Color Resolution	41
5.5	Action Schema and Supported Actions	42
5.6	Feature Coverage	43
6	Evaluation	45
6.1	Automated Test Suite	45
6.2	Experimental Setup	46
6.3	Multi-Model Evaluation Results	48
6.4	Per-Category Analysis	51
6.5	Latency and Cost Analysis	53

6.6	Rule-Based Baseline Reference	54
6.7	Few-Shot Direct Code Generation Baseline	56
6.8	Error Analysis	57
7	Discussion	59
7.1	Explainability and Transparency	59
7.2	Educational Value	59
7.3	Model Selection Insights	60
7.4	Limitations	60
8	Conclusion and Future Work	62
8.1	Conclusion	62
8.2	Future Work	62
	Appendix	67
	Appendix A: Example Workflow Trace	67
	Appendix B: Sample LLM Prompt (Abridged)	69
	Appendix C: Folder Structure	70

Περίληψη

Η παρούσα πτυχιακή εργασία παρουσιάζει ένα σύστημα παραγωγής κώδικα βασισμένο σε Large Language Models (LLM), το οποίο μετατρέπει εντολές φυσικής γλώσσας σε κώδικα Python για την κατασκευή απλών τρισδιάστατων σκηνών σε ένα πλαίσιο υπολογιστικών γραφικών. Το σύστημα απευθύνεται σε αρχάριους χρήστες και φοιτητές που μαθαίνουν τον προγραμματισμό γραφικών, με στόχο όχι μόνο την παραγωγή λειτουργικού κώδικα και οπτικών αποτελεσμάτων, αλλά και την ανάδειξη του παραγόμενου κώδικα, της ιεραρχίας της σκηνής και των μετασχηματισμών αντικειμένων που διέπουν την τελική σκηνή.

Ο χρήστης μπορεί να περιγράψει μια σκηνή ή μια τροποποίησή της μέσω απλών εντολών φυσικής γλώσσας, όπως δημιουργία αντικειμένων, αλλαγή χρωμάτων, μετακίνηση στοιχείων, κλιμάκωση αντικειμένων ή προσθήκη φωτισμού. Το σύστημα αναλύει την εντολή, τη μετατρέπει σε δομημένη ενέργεια σκηνής, την επικυρώνει, παράγει προεπισκόπηση του αποτελέσματος και εν τέλει παράγει αναγνώσιμο κώδικα Python γραφικών που μπορεί να επιθεωρηθεί και να επεξεργαστεί. Παράλληλα, εκθέτει την ιεραρχία της σκηνής και την ενδιάμεση αναπαράσταση JSON, με σκοπό να βοηθήσει τους χρήστες να κατανοήσουν πώς η εντολή τους αντιστοιχεί σε βασικές έννοιες υπολογιστικών γραφικών.

Η αξιολόγηση του συστήματος πραγματοποιήθηκε με 146 εντολές που καλύπτουν δημιουργία αντικειμένων, μετασχηματισμούς, υλικά, φωτισμό, κατασκευή σκηνών βάσει pre-fabs, κινούμενα αντικείμενα και κίνηση τροχιάς, χρησιμοποιώντας τέσσερα LLM backends (GPT-4o-mini, GPT-4.1-mini, Gemini-2.5-Flash, Gemini-2.5-Flash-Lite) και ένα baseline βασισμένο σε κανόνες. Τα αποτελέσματα δείχνουν ότι το πρωτότυπο επιτυγχάνει ακρίβεια πρόθεσης 94,5% στο σύνολο των 146 εντολών και 19 κατηγοριών. Το μοντέλο με το χαμηλότερο κόστος (Gemini-2.5-Flash-Lite, \$0,064 ανά 100 εντολές) σημείωσε την υψηλότερη επίδοση στην αξιολόγηση, γεγονός που υποδηλώνει ότι το μέγεθος του μοντέλου ενδέχεται να μη συσχετίζεται γραμμικά με την απόδοση σε εργασίες παραγωγής αυστηρά δομημένης εξόδου, αν και το εν λόγω συμπέρασμα βασίζεται σε μία μόνο εκτέλεση αξιολόγησης.

Acknowledgements

I would like to thank my supervisors, Prof. Georgios Papagiannakis and Dr. Manos Kamarianakis, for their guidance and support throughout this project. I also thank the Department of Computer Science of the University of Crete for providing the academic environment that made this work possible.

List of Abbreviations

LLM	Large Language Model
NL	Natural Language
API	Application Programming Interface
IR	Intermediate Representation
ECS	Entity-Component-System
HITL	Human-in-the-Loop
JSON	JavaScript Object Notation
OBJ	Wavefront Object (3D file format)
USD	Universal Scene Description
GLSL	OpenGL Shading Language
MVP	Model-View-Projection
IPC	Inter-Process Communication
VAO	Vertex Array Object
VBO	Vertex Buffer Object
XAI	Explainable Artificial Intelligence
FID	Frochet Inception Distance
RLHF	Reinforcement Learning from Human Feedback
SotA	State of the Art

Abstract

This thesis presents a Large Language Model (LLM)-based code generation system that converts natural language prompts into Python code for constructing simple 3D scenes in a computer graphics framework. The system is designed for beginners and students who are learning graphics programming, aiming not only to produce working code and visual results, but also to expose the generated code, scene hierarchy, and object transformations behind the final scene.

Users can describe a scene or a scene modification using simple prompts, such as creating objects, changing colors, moving elements, scaling objects, or adding lights. The system parses the prompt, converts it into a structured scene action, validates it, previews the result, and generates readable Python graphics code that can be inspected and edited. It also exposes the scene hierarchy and intermediate JSON action, with the intention of helping users trace how their prompt maps to basic graphics concepts.

The system was evaluated using prompts covering object creation, transformations, materials, lighting, prefab-based scene construction, animations, orbit motion, and custom model loading, across four LLM backends (GPT-4o-mini, GPT-4.1-mini, Gemini-2.5-Flash, Gemini-2.5-Flash-Lite) and a rule-based baseline. On the original 125-command benchmark (15 categories), the prototype achieves up to 99.2% intent accuracy. The evaluation was subsequently extended to 146 commands across 19 categories to cover the new animation and orbit features. On the full set, the lowest-cost model (Gemini-2.5-Flash-Lite, \$0.064 per 100 commands) achieves 94.5% intent accuracy, remaining the top-performing model in the evaluation. The finding that a smaller, cheaper model outperforms larger alternatives on this constrained task suggests that model size may not linearly correlate with performance on highly constrained structured-output tasks, though this is based on a single evaluation run.

1 Introduction

1.1 Motivation

Beginners in computer graphics often struggle to translate high-level scene ideas into working graphics code. Even simple scenes require understanding objects, transformations, materials, lighting, hierarchy, and the structure of the chosen programming framework. A student wishing to place a red cube on top of a blue sphere must first understand 3D coordinate systems, homogeneous transformation matrices, parent-child entity relationships, material properties, and the API of the chosen framework, all before writing a single line of meaningful scene logic.

Professional tools such as Blender [5] or Unity [21] expose these concepts through rich graphical interfaces, but their interfaces and workflows can be overwhelming for students in an introductory graphics course. The conceptual gap between what a student imagines and what they can produce in code remains a significant barrier to entry.

Recent AI-based tools make 3D content creation more accessible by allowing users to describe scenes with natural language. Systems such as SceneCraft [11] generate Blender Python code from text descriptions, and tools such as DALL-E 3 [4] or generative diffusion models [20] produce 2D images from text. However, many of these tools behave like black-box generators: the user receives a final scene or image, but does not necessarily understand how the scene was constructed, how the code was generated, or how the result maps to the underlying graphics framework. When the goal is education rather than production, this opacity is counterproductive.

1.2 Research Question

The research question of this thesis is: **how can LLM-based code generation translate natural language prompts into editable Python computer graphics code while preserving transparency for educational use?**

This thesis approaches this question by using natural language as an entry point for learning rather than as a replacement for programming. The goal is to help students move from high-level intent to visible scene structure and readable Python graphics code, exposing intermediate representations that make the mapping from intent to implementation explicit.

1.3 Contributions

The main contributions of this thesis are:

1. **A structured natural-language-to-scene pipeline using an Intermediate Representation (IR)**, enabling deterministic Python code generation with structured validation of LLM output, unlike direct LLM code synthesis which is probabilistic and prone to API hallucinations. Every action the LLM produces is checked for required fields and allowed values before being applied to the scene; this does not eliminate all possible runtime errors but reduces classes of field-level errors that would otherwise surface as Python exceptions.
2. **An explainable scene-editing workflow** with transparent JSON action objects, a preview-apply mechanism, and a full undo history stack. Students can inspect every stage of the pipeline from natural language to JSON action to scene IR to Python code.
3. **A runtime bridge architecture** with all state visible on disk as JSON files, making every intermediate state auditable and debuggable.
4. **Support for external 3D assets**: OBJ and USD/USDA/USDZ files are loaded via natural language commands. Asset geometry is parsed at code-generation time and embedded as NumPy arrays, requiring no runtime asset loader.
5. **A multi-model evaluation** (up to 146 commands, 19 categories, 4 LLM backends + rule-based baseline) showing up to 99.2% intent accuracy on the 125-command benchmark and 94.5% on the full 146-command set. A separate automated unit/integration test suite (464 tests, 100% pass rate) validates software correctness independently.
6. **A cost-accuracy observation**: in this structured-output benchmark, the lowest-cost model evaluated (Gemini-2.5-Flash-Lite, May 2026 pricing) achieves the highest intent accuracy on both the 125-command benchmark (99.2%) and the extended 146-command set (94.5%), suggesting that lower-cost models may be competitive with or outperform larger models on highly constrained generation tasks.

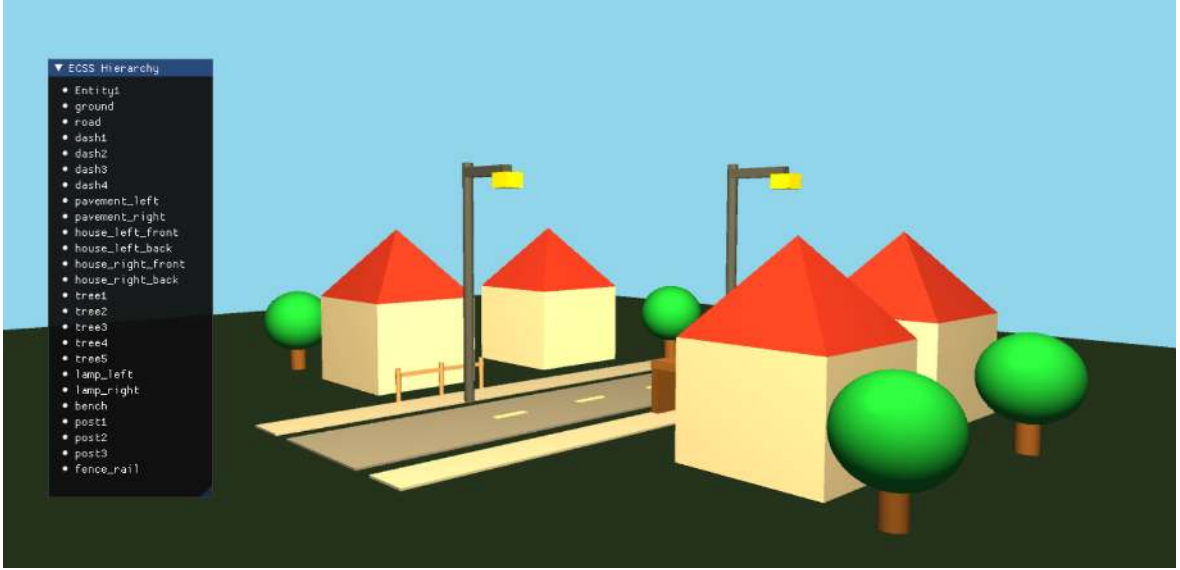


Figure 1: Example neighborhood scene produced by `textToScene` through a sequence of natural language commands. The scene includes four prefab houses, trees, lamp posts, a road with markings, a bench, and a ground plane – all reflected in the ECSS Hierarchy panel on the right.

1.4 Scope and Limitations

The system targets beginner-level, educational 3D scene construction. It is not designed for photorealistic rendering, large-scale scene synthesis, or physics simulation. It targets the Elements framework [18]. Evaluated scene complexity is restricted to primitive meshes, simple prefabs, and externally provided OBJ/USD assets at a single level of complexity.

1.5 Structure of this Thesis

The rest of this thesis is structured as follows. Section 2 provides background on Large Language Models, computer graphics fundamentals, the Entity-Component-System architecture, and the Elements framework. Section 3 reviews related work in text-to-3D generation, language-based scene editing, and LLM-based code generation. Section 4 describes the design decisions, system pipeline, intermediate representation schema, all major system components, and the dynamic object motion system covering animations and orbit. Section 5 covers implementation details including the shader system, texture support, lighting model, and the full action schema. Section 6 presents the automated test suite and multi-model evaluation results. Section 7 discusses explainability, educational value, model selection insights, and limitations. Finally, Section 8 concludes the thesis and outlines directions for future work.

2 Background

2.1 Large Language Models

Large Language Models (LLMs) are neural networks trained on massive corpora of text using a self-supervised learning objective. The dominant architecture since 2017 is the Transformer [23], which uses self-attention mechanisms to model long-range dependencies in sequences. Modern LLMs such as GPT-4 (OpenAI), Gemini (Google DeepMind), Claude (Anthropic), and LLaMA (Meta) are autoregressive: they predict the next token given all preceding tokens.

These models have demonstrated remarkable emergent capabilities, including instruction following, code generation, and structured output production. Critically for this work, LLMs can produce valid JSON when prompted with structured constraints, a capability that underpins the entire `textToScene` parsing pipeline.

Instruction-tuned models are fine-tuned with Reinforcement Learning from Human Feedback (RLHF) to follow user instructions. GPT-4o-mini and Gemini-2.5-Flash-Lite are instruction-tuned models optimized for cost and latency. Both expose an API that accepts a text prompt and returns a text response, which `textToScene` uses to convert natural language into structured JSON action objects.

Structured output is the ability of an LLM to return machine-parseable formats like JSON instead of free-form prose. This is achieved either through careful prompt engineering or through provider-level JSON-mode constraints. The design choice made in `textToScene` is discussed in Section 3.5.

Token economics are important for interactive systems. API pricing changes frequently and varies by tier. The full cost analysis for all evaluated models is presented in Section 6.5.

2.2 Computer Graphics Fundamentals

3D coordinate systems use three axes (X, Y, Z) to position objects in space. The Y axis typically points upward. Object position is described as a 3D vector $[x, y, z]$. `textToScene` uses Y-up convention throughout.

Homogeneous coordinates extend 3D vectors to 4D by adding a w component (typically 1 for positions, 0 for directions). This allows translations, rotations, and scales to all be expressed as 4×4 matrix multiplications, enabling them to be composed by matrix multiplication.

The **Model-View-Projection (MVP) transform** pipeline is the core of real-time

3D rendering:

- **Model matrix:** transforms object-local coordinates into world space. It encodes position, rotation, and scale.
- **View matrix:** transforms world coordinates into camera (eye) space, effectively moving the world so the camera is at the origin.
- **Projection matrix:** transforms from camera space to clip space, applying perspective (objects farther away appear smaller).

In `textToScene`'s generated code:

```
model = util.translate(x, y, z)
mvp = projMat @ view @ model
shader.setUniformVariable(key='modelViewProj', value=mvp, mat4=True)
```

Scene graphs organize 3D objects in a hierarchical tree structure. Parent transformations affect all children. `textToScene`'s IR directly encodes this hierarchy: the root is the scene node, its children are mesh objects or group nodes, and groups can contain further children.

Phong shading is a local illumination model computing per-fragment color as the sum of ambient, diffuse, and specular components. `textToScene` uses Phong shading via GLSL vertex and fragment shaders embedded in the generated script. The Phong equation is:

$$I = I_a K_a + I_d K_d \max(0, \mathbf{N} \cdot \mathbf{L}) + I_s K_s \max(0, \mathbf{R} \cdot \mathbf{V})^n \quad (1)$$

where I_a , I_d , I_s are ambient, diffuse, and specular light intensities; K_a , K_d , K_s are material coefficients; $\mathbf{N}$ is the surface normal; $\mathbf{L}$ is the light direction; $\mathbf{R}$ is the reflection vector; $\mathbf{V}$ is the view direction; and n is the shininess exponent.

2.3 Entity-Component-System Architecture

The **Entity-Component-System (ECS)** pattern is a composition-over-inheritance design pattern used extensively in game engines (Unity, Unreal Engine) and educational frameworks like Elements. It separates:

- **Entities:** Lightweight identifiers (essentially a unique ID and a name). They have no data or behavior of their own.

- **Components:** Plain data containers attached to entities. Examples include `BasicTransform` (stores the TRS matrix), `RenderMesh` (stores vertex arrays), and `ShaderGLDecorator` (stores shader program and uniforms).
- **Systems:** Behavior units that iterate over entities possessing specific components. Examples include `TransformSystem` (propagates transforms down the hierarchy), `CameraSystem`, and `RenderGLShaderSystem`.

The ECS architecture enables efficient data-oriented processing and clean separation of concerns. In `textToScene`’s generated code, every 3D object corresponds to four ECS declarations covering entity creation, hierarchy placement, transform attachment, and mesh/shader binding. A full annotated example is provided in Appendix A.

2.4 The Elements Framework

Elements (available at <https://github.com/papagiannakis/Elements>) is an educational Python graphics framework developed at the University of Crete [18]. It is built on OpenGL 4.x and PySDL2, and is used for computer graphics courses and curricula for undergraduate students.

Figure 2 shows the Elements framework in use.

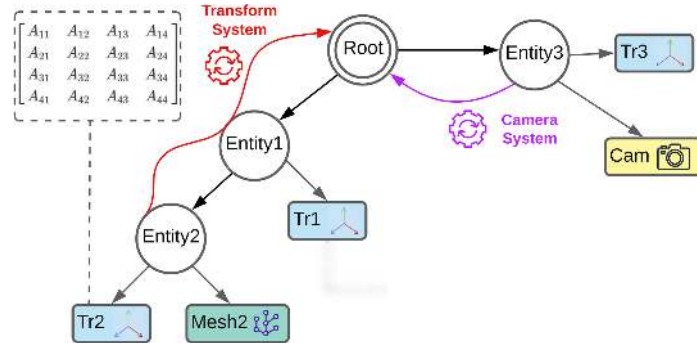


Figure 2: The Elements framework: an ECS-based Python graphics environment used for computer graphics education at the University of Crete. Source: [18].

Key features relevant to `textToScene`:

- **ECS architecture** (described in Section 2.3).
- **Explicit rendering:** there is no hidden scene-graph traversal or automatic optimization. Every render call is visible in user code.
- **ImGui integration:** `ImGuiEcssDecorator2` provides an in-scene editing panel rendered via Dear ImGui. `textToScene` uses this to embed the NL command input UI inside the running scene.

- **Built-in shader library:** Elements provides `Shader.VERT_PHONG_MVP` and `Shader.FRAG_PHONG` as standard Phong shading programs.
- **Math utilities:** `util.translate()`, `util.scale()`, `util.lookat()`, `util.perspective()` etc. wrap numpy operations for common graphics math.
- **Texture support:** `Elements.pyGLV.GL.Textures.Texture` loads image files and returns a texture object for shader binding.

`textToScene` generates code that imports directly from `Elements`. Running the generated scripts requires a full `Elements` installation. Importing USD assets additionally requires `usd-core`; you may need to run:

```
pip install usd-core
```

The generated scripts are not self-contained in terms of dependencies. They assume an environment where Elements and its prerequisites are installed.

Figure 3 shows the Scene Editor in preview mode after a simple natural language command.

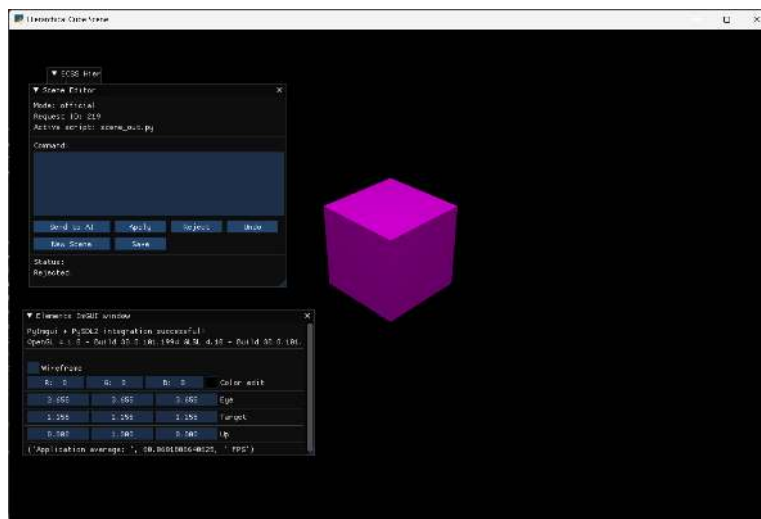


Figure 3: Scene Editor in preview mode after “add a purple cube” – the ECSS Hierarchy shows the new entity, and the panel exposes the full workflow buttons (Send to AI, Apply, Reject, Undo). Status reads “Preview ready.”

2.5 OpenGL and Shader Programming

OpenGL (Open Graphics Library) is a cross-platform API for hardware-accelerated 2D and 3D rendering. Elements targets OpenGL 3.x and 4.x. Key OpenGL concepts used in generated code:

- **Vertex Array Objects (VAO)** store the configuration of vertex attribute pointers. `textToScene` generates a `VertexArray` component for each mesh.
- **Vertex Buffer Objects (VBO)** store per-vertex data (positions, colors, normals, UVs) on the GPU.
- **Shader programs** are written in GLSL (OpenGL Shading Language) and run on the GPU. `textToScene` uses two shaders: the Phong MVP shader (for colored primitives) and a custom texture shader (for UV-mapped objects).

GLSL shaders in `textToScene`:

The Phong vertex shader (`VERT_PHONG_MVP`) transforms vertex positions through MVP and passes normals to the fragment stage. The fragment shader (`FRAG_PHONG`) computes Phong shading per fragment. For textured objects, `textToScene` embeds a custom two-stage GLSL shader (vertex + fragment) directly in the generated script. The full source is listed in Section 5.1.

3 Literature Review

3.1 Early Text-to-3D Work

Natural language has increasingly been explored as an interface for 3D scene generation and editing. Early work by Chang et al. [8] showed that spatial knowledge can be learned from text and used to generate 3D scenes from semantic descriptions, establishing an important connection between language and object placement in 3D environments. Their system used a probabilistic model trained on text-spatial correlations to place objects in a room given a text query.

3.2 Neural 3D Generation Systems

DreamFusion [19] introduced score distillation sampling (SDS) to optimize a NeRF representation using a 2D diffusion model as a loss function, enabling text-to-3D generation without 3D training data. This approach produces visually impressive results but is computationally expensive and produces opaque implicit representations (NeRFs) with no accessible geometry or code.

Magic3D [13] improved on DreamFusion by using a coarse-to-fine approach: first generating a coarse NeRF with SDS, then optimizing a textured mesh. This makes the output more useful for downstream applications, but still does not produce editable code.

Point-E [15] and **Shap-E** [12] generate 3D point clouds and implicit neural representations from text using conditional diffusion models. These systems are fast but produce non-editable output formats.

One-2-3-45 [14] uses multi-view diffusion to generate a 3D mesh from a single input image. The output is a mesh file, not code.

All of these neural generation systems prioritize visual quality over interpretability. None produce editable code or expose a scene hierarchy.

3.3 Language-Based Scene Editing

ReSpace (Bucher & Armeni, 2025) [7] introduces a text-driven framework for 3D indoor scene synthesis and editing, supporting operations such as object addition, removal, and replacement through natural language. ReSpace uses preference alignment with human annotators to rank candidate scene layouts. Its primary metric is scene plausibility judged by human annotators and a learned room-layout scorer, not code

correctness or action intent accuracy. ReSpace focuses on realistic indoor scene editing for computer vision researchers rather than beginner learners, and does not expose code or a structured IR.

SceneTeller (Öcal et al., 2024) [16] demonstrates language-based 3D scene generation and stylization for indoor environments. It generates scenes using a two-stage pipeline: first layout planning from text, then stylization. Evaluation uses FID (Frechet Inception Distance) on rendered images. It does not generate inspectable code or expose a scene hierarchy, and provides no educational transparency layer.

InstructNeRF2NeRF [10] uses text instructions to edit existing NeRF scenes by propagating 2D image edits into the 3D representation. This allows natural language editing of photo-realistic scenes but produces no code and is not interpretable.

3.4 Code Generation for Graphics

SceneCraft (Hu et al., 2024) [11] is the closest related work: it also generates Blender Python code from text, uses an LLM agent, and evaluates scene construction quality. SceneCraft targets autonomous high-fidelity scene synthesis for computer vision researchers, not beginner learners. Key differences from **textToScene** are:

- SceneCraft’s LLM directly emits executable Blender bpy code, so errors appear as Python runtime exceptions with no intermediate approval step.
- It does not expose an intermediate representation: the LLM is both the parser and the code generator.
- It does not provide an undo mechanism or preview/apply workflow.
- It does not target an educational use case.

Codex [9] and related work on LLM-based program synthesis [3] demonstrated that LLMs trained or fine-tuned on code can generate Python functions from docstrings with high pass rates on coding benchmarks. **textToScene**’s approach differs by not generating Python directly from LLM output. Instead, it uses the LLM only for structured JSON parsing, then generates Python deterministically from the IR.

3.5 Prompt Engineering and Structured Output

Chain-of-Thought prompting [24] improves LLM reasoning by asking models to explain their reasoning before giving a final answer. **textToScene** does not use chain-of-thought for the main parsing task (the latency would be too high for interactive

use), but the composite object generation sub-prompt does instruct the model to “work through the layout step-by-step” before writing JSON.

Few-shot prompting [6] shows the model multiple input-output examples in the prompt context, dramatically improving performance on structured tasks. `textToScene`’s main prompt includes approximately 30 JSON action examples covering all supported action types. This is a form of few-shot prompting without explicit input-output labels. It demonstrates the output format rather than mapping specific inputs to outputs.

JSON mode / structured outputs are provider-level features in OpenAI and Google APIs that constrain the LLM to produce valid JSON. `textToScene` uses prompt-level enforcement (instructing the model to return only JSON) rather than API-level constraints, for maximum portability across providers. The 100% parse success rate across all evaluated models in this study suggests that prompt-level enforcement is viable for this highly constrained task.

3.6 Explainable AI in Education

Explainable AI (XAI) refers to techniques for making AI systems’ decisions interpretable to humans [1]. In educational contexts, XAI is especially important: students need to understand not just what the system produced, but how and why, so they can learn from the process rather than just copy the output.

Human-in-the-loop (HITL) design involves keeping humans actively involved in the AI decision-making loop. `textToScene`’s preview/apply workflow is a form of Human-in-the-Loop (HITL) design: the system proposes a scene change and requires explicit human approval before committing it.

Intelligent Tutoring Systems (ITS) have long explored how to provide students with transparent feedback on their learning [22]. `textToScene` is not an ITS in the traditional sense, but shares the goal of exposing intermediate reasoning steps (the JSON action, the IR node, the generated Python code) to help students understand the mapping from intent to implementation.

3.7 Comparative Summary

Table 1: Comparison of `textToScene` with related systems.

Property	SceneCraft	ReSpace	SceneTeller	textToScene (ours)
Output type	Blender Python code	3D scene layout	3D scene + style	Elements Python code
Intermediate representa- tion	None	Scene graph	None	JSON IR (human- readable)
Preview/Apply workflow	No	No	No	Yes
Undo support	No	No	No	Full history stack
Educational focus	No	No	No	Yes
Asset import (OBJ/USD)	Limited	No	No	Yes
Eval metric	Scene quality	Human + scorer	FID	Intent accuracy (binary)
Cost-optimal model reported	N/A	N/A	N/A	Gemini-2.5- Flash-Lite

The gap addressed by this thesis is the absence of educational transparency in LLM-based graphics code generation: existing systems prioritize visual output over inspectable process. `textToScene` exposes the scene hierarchy, intermediate JSON action, and generated Python code at each step, targeting beginner learners rather than production use cases.

Figure 4 contrasts the black-box generation approach with the transparent pipeline of `textToScene`.

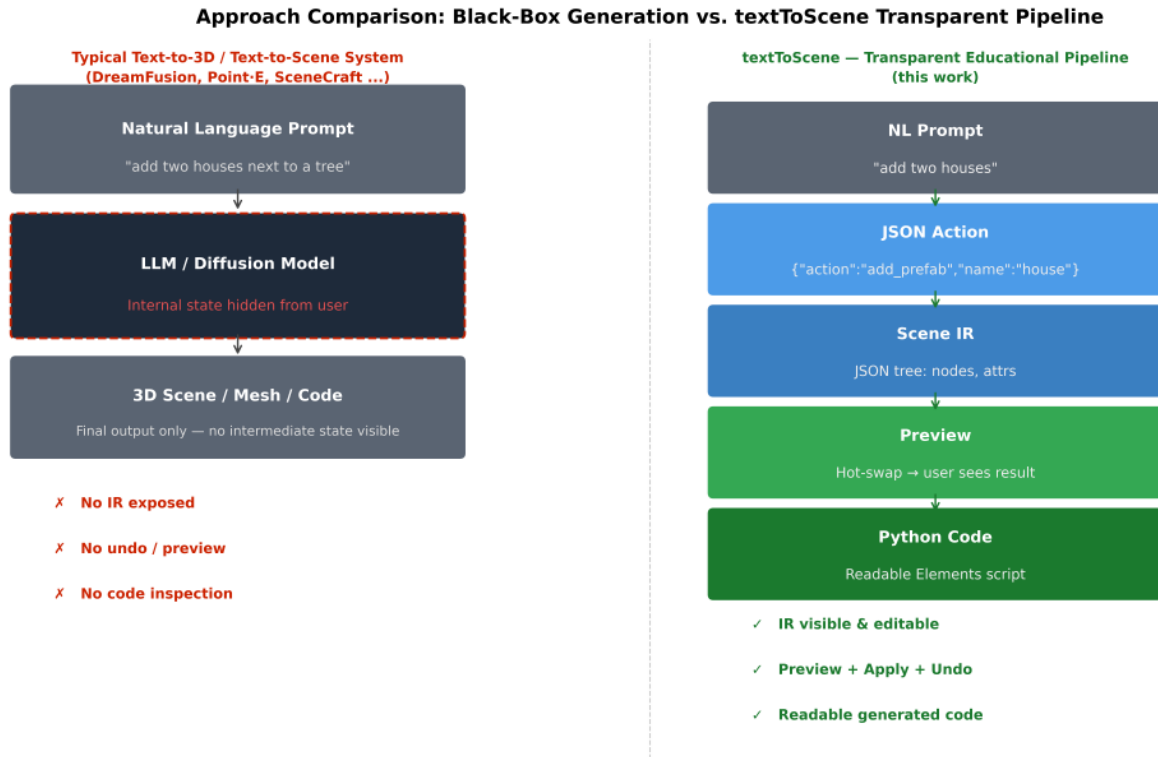


Figure 4: Black-box generation (DreamFusion, Point · E, SceneCraft) vs. transparent pipeline (textToScene): intermediate steps are hidden in the former and exposed in the latter.

4 Methodology

4.1 Target Framework: Elements

The Elements framework is described in Section 2.4. `textToScene` generates standalone Python scripts that use the Elements API directly. OBJ and USD assets are parsed at code-generation time and embedded as NumPy arrays, so no separate runtime asset-loader library is needed; however, running the scripts still requires a working Elements environment, and loading USDZ/binary USD assets at generation time additionally requires `usd-core`.

4.2 System Design Decisions

Several key design decisions shaped the architecture of `textToScene`:

Decision 1: Intermediate Representation over direct LLM code generation.

An alternative approach would be to ask the LLM directly for Python/Elements code (the approach taken by SceneCraft for Blender). This was rejected for three reasons:

- LLMs hallucinate API calls. An LLM asked for Elements code might call functions that do not exist, use wrong argument types, or produce syntactically invalid Python.
- Direct code generation produces non-deterministic output: the same command may produce different code in different runs, making debugging and learning harder.
- There is no natural insertion point for validation or preview.

The chosen approach: LLM produces a structured JSON action, a deterministic code generator converts the scene IR to Python. This approach substantially mitigates all three problems. The IR acts as a structural checkpoint between LLM output and the final script, catching field-level errors before code is generated. It does not eliminate the possibility of semantically incorrect but structurally valid actions.

Decision 2: File-based runtime bridge over in-process communication.

All state (scene IR, AI requests, UI actions) is stored as JSON files in a shared directory. This means:

- Every intermediate state is human-readable and auditable.

- The controller, scene script, and supervisor can be separate processes with no shared memory.
- Debugging is straightforward: open the JSON file to see exactly what the system believes the scene looks like.

Decision 3: Preview before apply.

Any command that modifies the scene first produces a preview: a separate Python script showing the proposed change is launched while the original scene remains active. The user must explicitly click “Apply” to commit the change. This is a fundamental HITL (Human-in-the-Loop) mechanism, the user inspects the result before accepting it, and can reject it without affecting the canonical scene.

Decision 4: Cumulative scene context in LLM calls.

Each LLM call includes a JSON summary of the current scene’s objects (names, IDs, shapes, colors, positions). This allows the LLM to resolve natural language references like “the red cube” or “the sphere I just added” to specific object names/IDs in the scene. Without context, placement commands (“put it on top of the cube”) would be impossible to resolve.

4.3 System Pipeline

The full pipeline has seven stages:

Table 2: System pipeline stages.

1. Prompt Input	The user writes a natural language prompt in the ImGui panel embedded in the running scene.
2. Parsing	The prompt is looked up in the local action cache; on a cache miss, it is sent to the LLM.
3. Structured Scene Action	The LLM returns a JSON action object describing the intended operation.
4. Validation	The action is validated against the IR schema (required fields, allowed values).
5. Preview / Apply	The system generates a preview scene with the proposed change and hot-swaps it in. The user approves or rejects.

Stage	Description
6. Scene Update	On apply, the canonical scene IR is updated.
7. Code Generation	The entire canonical scene IR is re-serialized to a complete Python Elements script.

Figure 5 provides a visual overview of these seven stages.

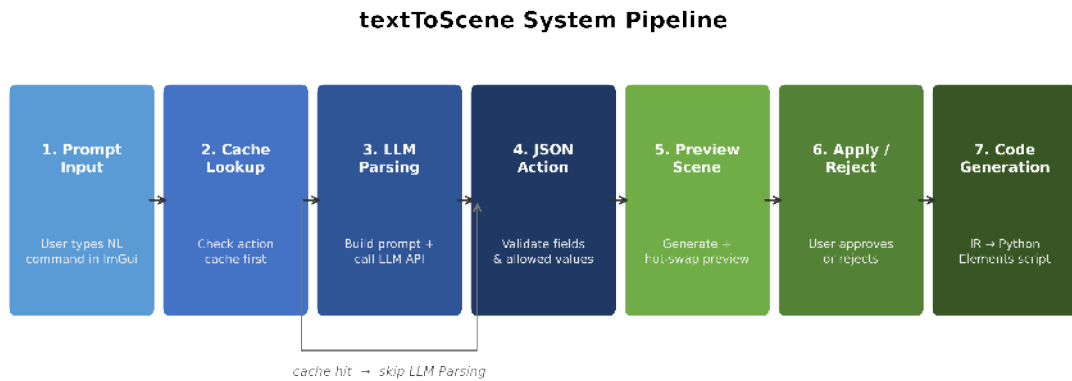


Figure 5: The seven-stage `textToScene` pipeline.

4.4 Intermediate Representation (IR) Schema

The IR is a JSON tree with the scene root as the top-level object. The schema supports four node types: `scene`, `mesh_object`, `group`, and `light`.

Scene node (root, always exactly one, a singleton):

```

{
  "node_type": "scene",
  "name": "root",
  "window": {
    "width": 1200,
    "height": 800,
    "title": "Generated Scene"
  },
  "children": [ ... ]
}

```

Mesh object node (a renderable primitive or custom asset):

```

{
  "node_type": "mesh_object",

```

```

    "name": "cube_1",
    "id": "cube_1",
    "created_order": 1,
    "shape": "cube",
    "transform": {
      "position": [0.0, 0.5, 0.0],
      "rotation": [0.0, 0.0, 0.0],
      "scale": [1.0, 1.0, 1.0]
    },
    "material": {
      "color": [0.8, 0.0, 0.0],
      "texture": {
        "enabled": false,
        "path": null
      }
    }
  }
}

```

Supported shapes: cube, sphere, cylinder, cone, pyramid, plane, custom (OBJ/USD). This list can be extended by adding new cases to the geometry factory and registering the new shape name in the IR schema.

Group node (a named container with a transform, parent of child mesh objects; used for prefabs and composite objects):

```

{
  "node_type": "group",
  "name": "tree_1",
  "transform": {
    "position": [2.0, 0.0, 0.0],
    "scale": [1.0, 1.0, 1.0]
  },
  "children": [
    { "node_type": "mesh_object", "name": "tree_1_trunk", ... },
    { "node_type": "mesh_object", "name": "tree_1_crown", ... }
  ]
}

```

Light node (point or directional light):

```

{
  "node_type": "light",
  "name": "light_1",
  "light_type": "point",
  "properties": {
    "position": [3.0, 5.0, 3.0],
    "color": [1.0, 1.0, 1.0],
    "intensity": 1.2
  }
}

```

```
}  
}
```

The IR schema is validated by a normalization function that traverses the tree and raises errors for missing or malformed fields. Default values are merged before this check, so partial actions from the LLM are completed with safe defaults where possible. This structural checking significantly reduces the chance of runtime failures, though it cannot guarantee that the resulting scene matches the user’s intent.

4.5 Prompt Engineering and LLM Integration

The system prompt sent to the LLM is assembled by the parser module and consists of:

1. **Role definition:** “You are a strict scene-editing parser.”
2. **Output format constraints:** “Return exactly one JSON object. Do not return markdown. Do not return explanations.”
3. **Allowed action list:** All 22 supported top-level action types.
4. **Action sequence rule:** Instructions for when to use `action_sequence` vs. a single action.
5. **Field rules:** Specific instructions for each action (rotation defaults, scale mappings, texture name whitelist, etc.).
6. **Allowed values:** Object types, direction values, axis values, texture names, prefab names.
7. **~30 JSON examples:** One or more concrete examples per action type covering all supported actions.
8. **User prompt:** The actual user command.
9. **Scene context:** A JSON summary of all current scene objects.

The prompt is approximately 6,300 tokens per call. This is relatively large; the trade-off is reliability: the extensive examples prevent the LLM from making format mistakes, and the field rules prevent common errors (e.g., using “rotation” instead of “degrees” for `rotate_object`).

Key prompt rules for robustness:

- `rotate_object` must use { `"axis": "x|y|z", "degrees": <number> }`. If the user does not specify an axis, default to “y”. If no amount, default to 45 degrees. This prevents the model from using alternative field names like `angle` or `rotation`.
- `scale_object` maps verbal descriptions: “make it bigger” becomes `factor: 1.5`, “make it smaller” becomes `factor: 0.66`, “double” becomes `factor: 2.0`, “half” becomes `factor: 0.5`.

- `apply_texture` must use only texture names from the catalogue: brick, wood, stone, grass, metal, sand, marble, concrete. This catalogue can be expanded by adding new image files to the `textures/` directory and registering their names in the schema.
- `add_custom_model` must use a `model_path` that is a filename (no directories) ending in `.usd` or `.obj`.
- The distinction between `add_prefab` (use when the user says “add a [object]” with no shape) and `generate_composite` (use when the user says “make a [object] with [shape]”).

Provider routing in `_call_llm()`:

```
def _call_llm(model_name, prompt_text):
    if str(model_name).lower().startswith("gemini"):
        return _call_gemini(model_name, prompt_text)
    return _call_openai(model_name, prompt_text)
```

The Gemini caller uses the `google-genai` SDK and implements exponential backoff for rate-limit errors (429 / `RESOURCE_EXHAUSTED`), waiting 30s, 60s, then 120s before failing. The OpenAI caller uses the `openai` SDK’s Responses API.

JSON extraction and normalization post-processes the raw LLM response:

1. Strip markdown code fences (````json ... ````).
2. Extract the first complete JSON object (from `{` to matching `}`).
3. Parse with `json.loads()`.
4. Apply `normalize_parsed_action()`: canonicalize field aliases (`angle` to `degrees`, `name` to `target` for `rotate_object`), fix action name aliases (`rotate` to `rotate_object`, `composite` to `action_sequence`).
5. Validate the `action` field is present and the `action_sequence` list is non-empty when relevant.

4.6 Action Cache Mechanism

LLM API calls have latency (~1.5–4 seconds) and cost. Many user commands are semantically identical to previous ones (“add a red cube”, “move the sphere right”). The action cache avoids redundant API calls for these cases.

Cache structure (`cache/action_cache.json`):

```
{
  "version": 1,
  "updated_at": 1717600000.0,
  "entries": {
    "add red cube": {
      "prompt": "Add a red cube",
      "normalized_prompt": "add red cube",
      "action": { "action": "add_object", "object_type": "cube", "
        color": "red" },
      "updated_at": 1717500000.0
    }
  }
}
```

Prompt normalization for cache lookup:

1. Convert to lowercase.
2. Replace punctuation and underscores with spaces.
3. Remove filler words: “a”, “an”, “the”, “please”.
4. Collapse whitespace.

So “Please add a red cube!” normalizes to “add red cube” and “Add the red cube” also normalizes to “add red cube”, meaning both hit the same cache entry.

Cacheability rules: Only actions that do not depend on current scene state are cached:

- **Cacheable:** `add_object` with no placement target, plus `generate_pattern` and `generate_composite`.
- **Not cached:** scene-dependent actions such as `move_object`, `delete_object`, `recolor_object`, because the same command may refer to different objects in different scenes.
- **Field-level check:** the cache also inspects action fields for scene-dependent keys (`id`, `object_id`, `position`, `new_position`) and skips caching if any are present.

Atomic file writes: The cache is written using a write-then-rename pattern (`tmp_path` renamed to `real_path`) to prevent corruption if the process is interrupted during a write.

4.7 Runtime Bridge Architecture

The runtime bridge coordinates three concurrent processes.

Controller: Polls `ai_request.json` for new requests. When a request arrives:

1. Parses the prompt (cache lookup first, then LLM on miss).
2. Applies the action to the current scene IR (`apply_action_to_ir()`).
3. Generates a preview Python script via `generate_scene_script()`.
4. Writes the preview IR and preview script.
5. Updates `ai_request.json` status to `preview_ready`.
6. Updates `scene_state.json` to point to the preview script.

Scene Script: A running Elements application. It embeds an ImGui panel that:

- Lets the user type commands.
- Polls `ai_request.json` and `scene_state.json` every frame.
- Shows status messages as the command progresses: pending, then preview ready, then applied or rejected.
- Sends UI events (apply, reject, undo, new_scene, save) by writing `ui_state.json`.

Supervisor: Monitors `scene_state.json` and `ui_state.json`. On “apply”, it copies the preview IR to the canonical IR, generates the official scene script, and restarts the scene process with the new script. On “reject”, it restores the official scene without changes. On “undo”, it pops the history stack and regenerates the previous scene.

JSON files used as IPC:

File	Purpose
<code>scene_ir.json</code>	Canonical scene Intermediate Representation
<code>preview_scene_ir.json</code>	IR of the staged (uncommitted) preview
<code>ai_request.json</code>	Current request status and parsed action
<code>ui_state.json</code>	User UI actions (apply, reject, undo)
<code>scene_state.json</code>	Lightweight scene metadata (mode, active script path)
<code>history/undo_stack.json</code>	File-based undo stack
<code>cache/action_cache.json</code>	Prompt-to-action cache

All JSON writes use atomic rename to prevent race conditions between concurrent readers and writers.

Figure 6 illustrates this three-process architecture and the JSON files used for inter-process communication.

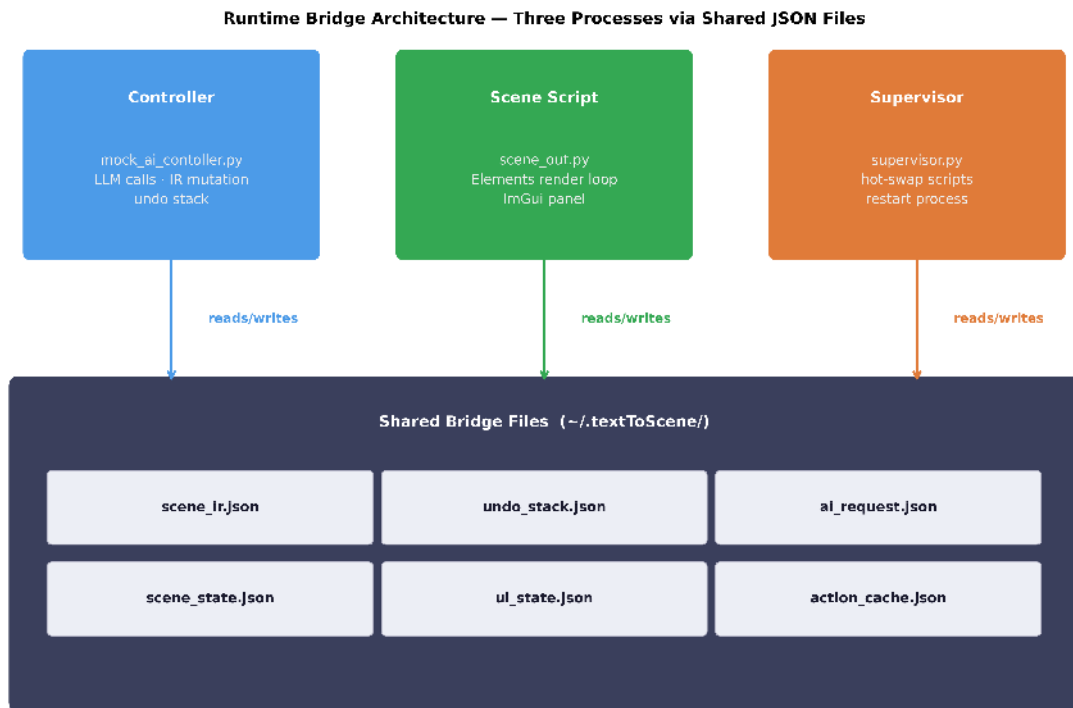


Figure 6: Runtime bridge architecture: three independent OS processes communicating exclusively via shared JSON files.

4.8 Preview / Apply / Undo Workflow

Preview is the core educational mechanism. When the user submits a command:

1. The controller computes the new scene IR (with the action applied) and generates `preview_scene.py`.
2. The supervisor hot-swaps the running scene script to `preview_scene.py`.
3. The user sees the proposed change in real time (the preview scene is fully rendered by Elements).
4. The user clicks “Apply” or “Reject”.

Apply:

1. `scene_ir.json` is updated with the preview IR.
2. A snapshot of the previous IR is pushed onto the undo stack (`history/undo_stack.json`).
3. The official `scene_out.py` is regenerated.
4. The supervisor hot-swaps back to the official script.

Reject:

1. The preview IR is discarded.
2. The supervisor hot-swaps back to the official script.
3. `scene_ir.json` is unchanged.

Undo:

1. The top entry of the undo stack is popped.
2. `scene_ir.json` is overwritten with the popped IR snapshot.
3. `scene_out.py` is regenerated from the restored IR.
4. The supervisor hot-swaps to the restored script.

The undo history is file-based, meaning it survives process restarts and can be inspected by the user. The stack is serialized as a JSON array of IR snapshots.

Figure 7 shows the Scene Editor panel in preview mode during this workflow.

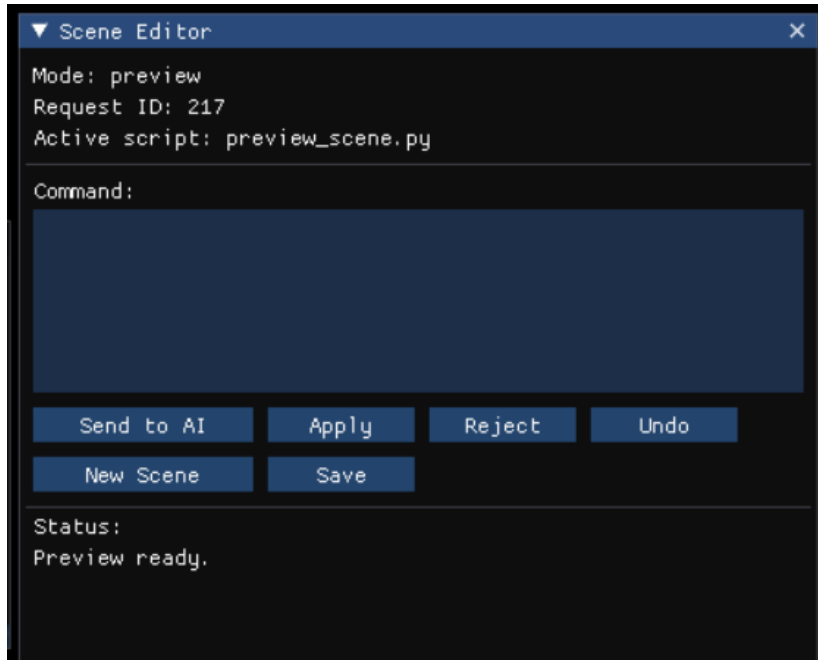


Figure 7: Preview/Apply/Undo workflow – Scene Editor panel in preview mode.

4.9 Python Code Generation Pipeline

The code generator (`code_generator.py`) is a purely deterministic function: given the same scene IR it always produces identical Python output. It traverses the IR tree depth-first and emits one Elements API call sequence per node.

Generation stages:

1. **Validate and normalize** the IR (merge defaults, validate field types).
2. **Collect lights** from the IR tree. Use the first light node found (if any) to configure `activeLightPos`, `activeLightColor`, `activeLightIntensity`.
3. **Build header**: import statements, shader string literals, scene setup (camera, view/projection matrices, systems).
4. **Emit object nodes**: depth-first traversal of the IR tree, producing geometry data variables and ECS declarations.
5. **Emit uniform updates**: for each mesh object, produce the per-frame uniform update code (MVP matrix, material color, light uniforms).
6. **Build footer**: ImGui panel code, render loop, poll/draw calls.

Header generation includes all Elements imports, the GLSL texture shader source code, camera setup, and the light configuration:

```
eye      = util.vec(2.5, 2.5, 2.5)
target  = util.vec(0.0, 0.0, 0.0)
```

```
up      = util.vec(0.0, 1.0, 0.0)
view    = util.lookat(eye, target, up)
projMat = util.perspective(50.0, winWidth / winHeight, 0.01, 100.0)
```

Geometry emission (`emit_geometry_data()`) calls the geometry factory to generate vertex, index, color, and normal arrays at code-generation time. These are embedded in the script as `np.array(..., dtype=np.float32)` literals. This means the generated script has no runtime dependency on the geometry factory and is fully self-contained.

Transform emission converts the IR transform to Elements API calls:

- **Position:** `util.translate(x, y, z)`
- **Scale:** `util.scale(s)` (uniform) or a 4×4 NumPy array (non-uniform)
- **Rotation:** `util.eulerAnglesToRotationMatrix(np.radians([rx, ry, rz]))`
- **Composition:** TRS matrices are combined as `translate @ rotate @ scale`

World transform for nested objects (groups) uses matrix composition:

```
world_trs = parent_trs @ local_trs
mvp = projMat @ view @ world_trs
shader.setUniformVariable(key='modelViewProj', value=mvp, mat4=True)
```

Footer generation produces the complete ImGui rendering loop, including the `draw_editor_panel()` function which embeds the NL command UI, and the main render loop:

```
while running:
    running = scene.render()
    scene.world.traverse_visit(renderUpdate, scene.world.root)
    scene.world.traverse_visit_pre_camera(camUpdate, orthoCam)
    scene.world.traverse_visit(camUpdate, scene.world.root)
    view = gWindow._myCamera
    # ... per-shader uniform updates ...
    poll_backend_state()
    draw_editor_panel()
    scene.render_post()
```

4.10 OBJ and USD Asset Import

Users can load external 3D assets via natural language: “Load the chair model from Koltuk.obj”. The controller resolves the model path relative to the bridge’s `custom_models/` directory and creates a `mesh_object` IR node with `shape:"custom"` and a `path` field for the asset.

At code generation time, the code generator parses the asset file and embeds the resulting numpy arrays directly in the Python script. No runtime loader is needed.

OBJ parsing (`_parse_obj_for_codegen()`):

1. Read vertex positions (**v** lines) and face indices (**f** lines).
2. Fan-triangulate polygonal faces: for an N-gon ($v_0, v_1, \dots, v_{N-1}$), generate triangles (v_0, v_1, v_2), (v_0, v_2, v_3), etc.
3. Compute per-vertex smooth normals by accumulating face normals and normalizing.
4. Apply the user-specified color to all vertices.
5. Return four numpy arrays: **vertices** ($N \times 4$ homogeneous), **indices** ($M \times 3$), **normals** ($N \times 3$), **colors** ($N \times 4$).

```
# OBJ parsing excerpt (simplified)
for raw_line in f:
    parts = line.split()
    if parts[0] == "v":
        positions.append([float(p) for p in parts[1:4]])
    elif parts[0] == "f":
        verts = [int(tok.split("/")[0]) - 1 for tok in parts[1:]]
        for k in range(1, len(verts) - 1):
            face_indices.append([verts[0], verts[k], verts[k + 1]])
```

USD (Universal Scene Description [17]) **parsing** (`_parse_usd_for_codegen()`): Two strategies depending on file format:

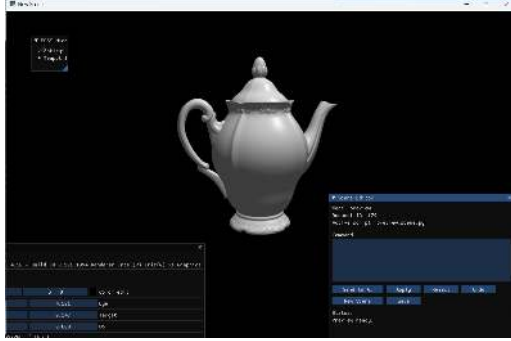
- **USDA (text USD)**: Parsed using pure regex without the **pxr** library. Extracts **points**, **faceVertexIndices**, and **faceVertexCounts** arrays directly from the ASCII text.
- **USD/USDZ (binary)**: Requires **usd-core** (`pip install usd-core`). Opens the stage via the **pxr** library, iterates all Mesh prims, applies world transforms and unit scaling (**metersPerUnit**), and normalizes the combined mesh to fit in a 2-unit bounding box.

Mesh normalization for USD: Large assets (e.g., from CAD software stored in centimeters) are automatically scaled to fit within a 2-unit bounding box:

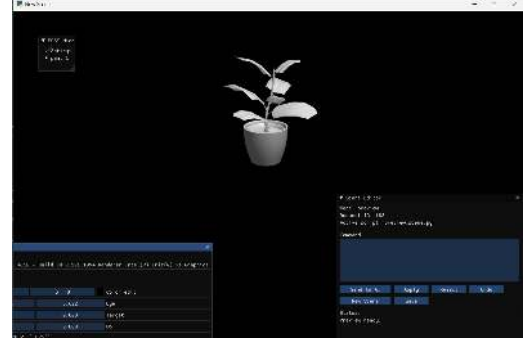
```
span = max(max(xs)-min(xs), max(ys)-min(ys), max(zs)-min(zs))
scale = target_size / span # target_size = 2.0 units
```

This ensures imported models appear at a reasonable scale in the scene regardless of their original unit system.

Figure 8 shows examples of USD and OBJ models imported using this pipeline.



(a) USD import – teapot model.



(b) USD import – plant model.



(c) Chameleon model.

(d) Teapot model.



(e) Ball model.

Figure 8: USD and OBJ models imported via the `textToScene` custom model pipeline.

4.11 3D Model Loading and Texture Rendering

Beyond procedurally-generated primitive shapes, the system supports loading external USDZ models with embedded textures. Three representative models are shown in Figure 8: a chameleon (Figure 8c), a baseball (Figure 8e), and a teapot (Figure 8d).

Each model is rendered using a custom Phong shading pipeline implemented directly in GLSL. The shader computes per-fragment ambient, diffuse, and specular contributions from a point light source. Smooth vertex normals are derived at load time from face cross-products, eliminating the faceted appearance of flat shading.

To ensure the texture shading is perceptible from all viewing angles, the system bakes a dynamic orbiting light into every textured scene script. The light position is updated each frame using `np.cos / np.sin` on `time.time()`, circling the model at a fixed radius. No additional user interaction is required to activate this effect.

Models are loaded by natural language command (“load a chameleon”, “add the teapot”) through the `detect_procedural_action` function, which matches the command against

a dictionary of bundled model keywords before invoking the LLM pipeline.

4.12 Geometry Factory

The geometry factory (`geometry_factory.py`) generates parametric mesh data (vertices, indices, colors, normals, UV coordinates) for each supported primitive shape. It implements two shading modes:

Flat shading (for hard-surface shapes: cube, pyramid, plane): Each triangle gets its own copy of the face normal, stored as a constant across all three vertices. This produces sharp, faceted appearance.

Smooth shading (for curved shapes: sphere, cylinder, cone): Vertex normals are averaged across all adjacent faces, producing smooth silhouettes.

The factory also generates **UV coordinates** for texture mapping, used when an object has a texture assigned. UV generation uses standard cylindrical/spherical/box projection depending on shape type.

Shape generation methods:

- **Cube:** 8 vertices, 12 triangles (2 per face), 6 faces. Flat shading. UVs assigned per face.
- **Sphere:** Parameterized by latitude/longitude subdivision (default 16×16). Smooth shading. Spherical UV mapping.
- **Cylinder:** Top and bottom caps (flat) + side surface (smooth). UV: cylindrical mapping for sides, polar for caps.
- **Cone:** Apex vertex + bottom cap. Side: smooth shading. UV: cylindrical.
- **Pyramid:** Square base + 4 triangular faces. Flat shading.
- **Plane:** Simple quad (2 triangles). Flat shading. UV: planar.

All geometry is pre-scaled to fit within approximately a 1-unit bounding box. The scene transform handles actual positioning and scaling.

4.13 Prefab System

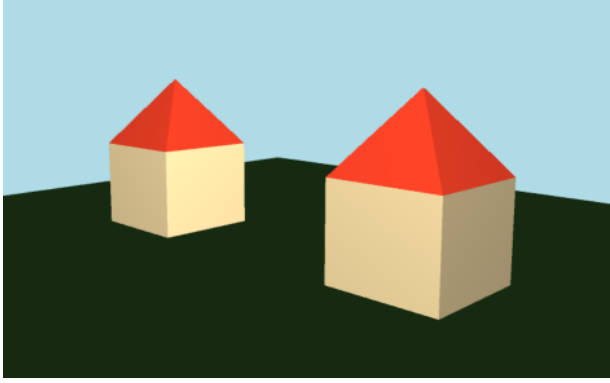
Prefabs are hand-authored multi-part objects defined in `prefabs.py`. Each prefab function takes a name and position and returns a group IR node with children. Available prefabs: house, tree, chair, bench, bed, table, lamp, street_light, gift_box. New prefabs can be added by defining a new builder function in `prefabs.py` and registering its name in the action schema.

Tree prefab example:

```
def build_tree(name, position):
    return {
        "node_type": "group",
        "name": name,
        "transform": {"position": position, "scale": [1.0, 1.0, 1.0]},
        "children": [
            {
                "node_type": "mesh_object",
                "name": f"{name}_trunk",
                "shape": "cylinder",
                "transform": {"position": [0.0, 0.22, 0.0], "scale": [0.18, 0.45, 0.18]},
                "material": {"color": [0.45, 0.28, 0.12]}
            },
            {
                "node_type": "mesh_object",
                "name": f"{name}_crown",
                "shape": "sphere",
                "transform": {"position": [0.0, 0.62, 0.0], "scale": [0.75, 0.75, 0.75]},
                "material": {"color": [0.1, 0.6, 0.2]}
            }
        ]
    }
```

Prefabs are triggered by the `add_prefab` action and are purely data. They do not contain code, making them fully serializable as IR JSON. The code generator handles group nodes the same way regardless of whether they came from a prefab, a composite, or a user-defined group.

Figures 9a and 9b show a two-house prefab scene and its expanded ECSS hierarchy; Figure 10 shows a more complex multi-prefab composition.



(a) Two prefab houses rendered after an `add_prefab` command. The ECSS Hierarchy shows `house1` and `house2` as top-level nodes.



(b) The same prefab house scene with the ECSS Hierarchy expanded, revealing the `_body` and `_roof` child nodes for each house.

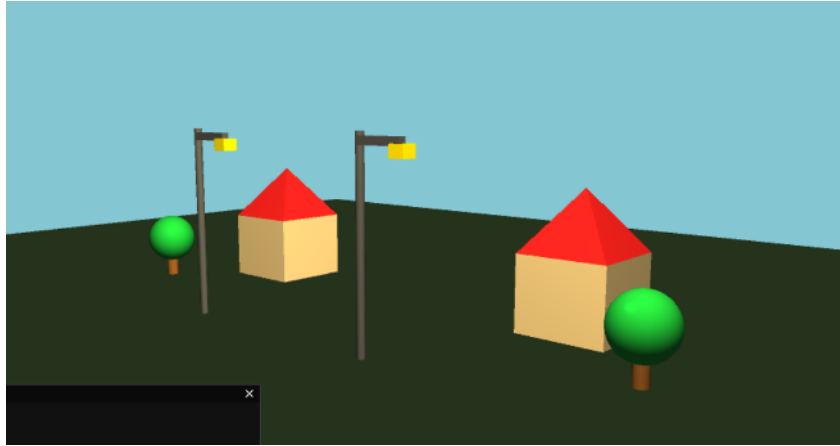


Figure 10: Complex prefab scene built incrementally through natural language commands, combining houses, trees, and a street light. The ECSS Hierarchy shows deep nesting: `trunk/crown` for trees and `pole/arm/lamp` for the street light.

4.14 Dynamic Object Motion

To extend the expressive capabilities of the system, three per-frame animation types were implemented: bounce, spin, and lerp (linear interpolation between two positions), along with an orbit mode that causes an object or light source to continuously circle a target object in the scene.

All motion commands are detected by the `detect_procedural_action` function before the LLM is invoked, using regular expressions that cover multiple natural-language variants of each motion type. For example, the commands “make the cube bounce”, “make the cube go up and down”, and “make the cube bob” all resolve to the same bounce action. This design eliminates API latency for motion commands and guarantees deterministic behaviour regardless of how the user phrases the request.

4.14.1 Orbit

Orbit is triggered by commands such as “add a sphere that orbits the cube” or “add a sphere that revolves around the cube”. The controller resolves the target object’s current world position from the scene IR and stores it as the orbit centre. At code-generation time, the orbiting object’s position is recomputed every frame using trigonometric functions:

```
_t = time.time()
x = cx + radius * np.cos(_t * speed)
z = cz + radius * np.sin(_t * speed)
model = util.translate(x, y, z) @ util.scale(sx, sy, sz)
```

This produces smooth, continuous circular motion in the XZ plane around the target, with no further user interaction required after the initial command.

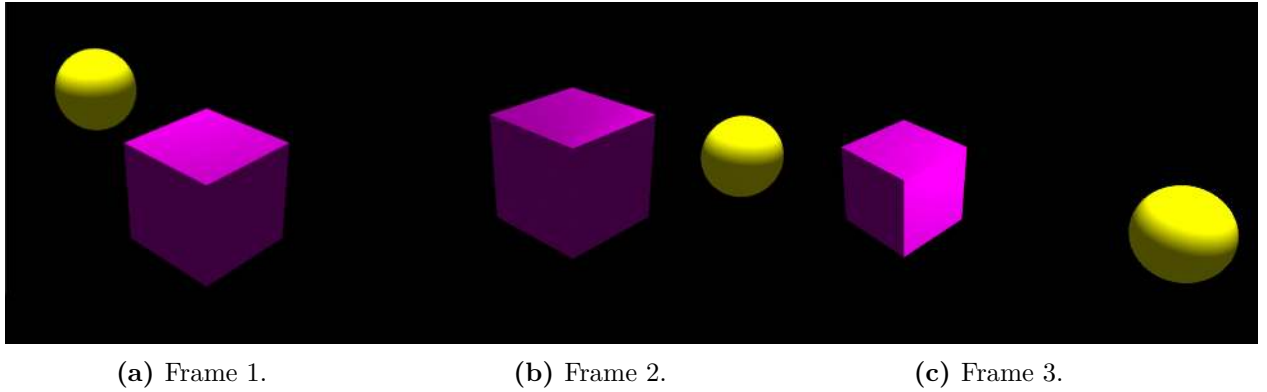


Figure 11: Three successive frames showing a sphere orbiting a cube, generated from the command “add a sphere that orbits the cube”.

Figures 11(a), 11(b), and 11(c) show three successive frames of the scene: the sphere appears at three distinct positions along its orbit around the cube, illustrating the continuity and smoothness of the motion.

4.15 Composite Object Generation

Composite generation is a two-LLM-call pipeline:

1. **Main LLM call** (main prompt): User says “make a lamp with cylinders”, and the LLM returns a `generate_composite` action with `"composite": "lamp"` and `"object_type": "cylinder"`.
2. **Composite spec call**: A second, focused prompt asks the LLM to design the 3D layout of the specified composite object using the specified primitive. This call is handled by `parse_composite_spec_with_llm()`:

You are a 3D scene layout designer. Describe a real-world object as a list of non-overlapping 3D primitive parts.

Object to design: lamp

Preferred primitive shape: cylinder

Return exactly one JSON object:

```
{
  "parts": [
    {"name": "base", "shape": "cylinder",
      "position": [x,y,z], "scale": [sx,sy,sz]},
    ...
  ]
}
```

Spatial rules:

1. No two parts may share interior volume.
2. Ground plane is $y=0$. Every part must satisfy $y \geq sy/2$.
3. To stack B on A: $B_y = A_y + A_{sy}/2 + B_{sy}/2$.

The composite spec response is validated (3–8 parts, required fields, positive scales) and converted into a group IR node. The controller then calls an overlap-resolution pass that checks for interpenetrating bounding boxes and adjusts y positions upward if needed.

4.16 Module Organization

The `textToScene` system is organized into eight Python source modules (under `src/`):

Table 4: Source modules of the `textToScene` system.

Module	Responsibility	Key Functions
<code>llm_parser.py</code>	LLM communication, prompt building, JSON parsing, action cache	<code>parse_prompt_to_action_with_llm()</code> , <code>build_prompt()</code> , <code>lookup_cached_action()</code>
<code>mock_ai_contoller.py</code>	Command dispatch, IR mutation, placement geometry, undo stack	<code>apply_action_to_ir()</code> , <code>find_position_on_top_of_target()</code> , <code>parse_command()</code>
<code>code_generator.py</code>	Deterministic IR-to-Python script translation	<code>generate_scene_script()</code> , <code>emit_mesh_object_node()</code> , <code>emit_group_node()</code>
<code>geometry_factory.py</code>	Parametric mesh generation (vertices, normals, UVs)	<code>make_cube()</code> , <code>make_sphere()</code> , <code>make_cylinder()</code> , <code>make_cone()</code>
<code>ir_schema.py</code>	IR schema validation and canonical default scene construction	<code>validate_scene_ir()</code> , <code>empty_scene_ir()</code>
<code>config.py</code>	All path constants, API keys, model selection, texture catalogue	<code>SHARED_DIR</code> , <code>ACTION_CACHE_FILE</code> , <code>DEFAULT_MODEL</code>
<code>supervisor.py</code>	Process management, script hot-swap on apply/reject/undo	<code>run_loop()</code> , <code>ensure_initial_state()</code> , <code>swap_to_official()</code>
<code>prefabs.py</code>	Hand-authored multi-part object definitions	<code>build_house()</code> , <code>build_tree()</code> , <code>build_lamp()</code> , etc.

5 Implementation Details

5.1 Shader System

`textToScene` uses two shader programs. The choice between them is made at code-generation time based on whether the object has a texture assigned.

Phong MVP Shader (default for colored primitives): This is Elements' built-in `Shader.VERT_PHONG_MVP` / `Shader.FRAG_PHONG`. Uniforms updated per frame:

```
shader.setUniformVariable(key='modelViewProj', value=mvp,
                           mat4=True)
shader.setUniformVariable(key='model', value=model,
                           mat4=True)
shader.setUniformVariable(key='ambientColor', value=Lambientcolor,
                           float3=True)
shader.setUniformVariable(key='ambientStr', value=Lambientstr,
                           float1=True)
shader.setUniformVariable(key='viewPos', value=LviewPos,
                           float3=True)
shader.setUniformVariable(key='lightPos', value=activeLightPos,
                           float3=True)
shader.setUniformVariable(key='lightColor', value=
    activeLightColor, float3=True)
shader.setUniformVariable(key='lightIntensity', value=
    activeLightIntensity, float1=True)
shader.setUniformVariable(key='shininess', value=Mshininess,
                           float1=True)
shader.setUniformVariable(key='matColor', value=mat_color,
                           float3=True)
```

Texture Shader (for objects with `texture.enabled = True`): A custom GLSL shader embedded as a string literal in the generated script:

```
// Vertex shader
layout (location=0) in vec4 vPos;
layout (location=1) in vec2 vTexCoord;
out vec2 fragmentTexCoord;
uniform mat4 model; uniform mat4 view; uniform mat4 proj;
void main() {
    gl_Position = proj * view * model * vPos;
    fragmentTexCoord = vTexCoord;
}

// Fragment shader
out vec4 outputColor;
in vec2 fragmentTexCoord;
```

```
uniform sampler2D texSampler;  
void main() { outputColor = texture(texSampler, fragmentTexCoord); }
```

Uniforms for texture shader are set per frame: model, view, proj matrices. The texture sampler is set once after initialization via `Texture(path)`.

5.2 Texture System

Texture mapping uses UV coordinates generated by the geometry factory and stored in `uv_<idx>` arrays in the generated script. The texture loading code is emitted in the post-init block (after `initUpdate` traversal):

```
texture_1 = Texture(r"/path/to/scene_bridge/textures/brick.jpg")  
shader_1.setUniformVariable(key='texSampler', value=texture_1,  
    texture=True)
```

The texture catalogue defines 8 built-in textures: brick, wood, stone, grass, metal, sand, marble, concrete. These are stored in `scene_bridge/textures/` and referenced by name in the action schema. The catalogue can be extended by adding new image files to that directory and registering their names in the schema.

When `apply_texture` is called, the IR is updated:

```
"material": {  
  "color": [1.0, 1.0, 1.0],  
  "texture": { "enabled": true, "path": "/full/path/to/brick.jpg" }  
}
```

When `remove_texture` is called, `texture.enabled` is set to false and the object reverts to Phong shading.

Figure 12 shows the result of applying a wood texture to a cube, demonstrating the reject flow.



Figure 12: User command: "apply wood texture to the cube". The Scene Editor shows the texture applied to `cube_tex`; the `Rejected` status indicates the user subsequently clicked `Reject`, reverting the scene to its previous state and demonstrating the reject workflow.

5.3 Lighting Model

`textToScene` supports two light types: **point lights** (omnidirectional from a position) and **directional lights** (parallel rays from a direction vector). Only one light is active at a time (the first light node in the IR tree). These defaults are hard-coded in the generated script but can be freely modified by the user in the output Python file.

Point light configuration:

```
activeLightPos      = util.vec(3.0, 5.0, 3.0)
activeLightColor    = util.vec(1.0, 1.0, 1.0)
activeLightIntensity = 1.2
```

For directional lights, the direction vector is converted to a pseudo-position (far away in the opposite direction) for compatibility with the Phong shader's `lightPos` uniform:

```
# direction [1, -1, -1] \textrightarrow{} pseudo-position [-10, 10, 10]
pseudo_pos = [-10.0 * d[0], -10.0 * d[1], -10.0 * d[2]]
```

Light actions supported: `add_light`, `delete_light`, `move_light`, `change_light_color`, `change_light_intensity`.

Ambient light parameters are fixed constants in the generated script:

```
Lambientcolor = util.vec(1.0, 1.0, 1.0)
Lambientstr   = 0.3
LviewPos      = util.vec(2.5, 2.8, 5.0)
Mshininess    = 0.4
```

5.4 Color Resolution

Colors are resolved in two stages. The LLM returns a color as a string (CSS color name or hex code). The controller resolves named colors to [R, G, B] float triples using a built-in color table defined in `mock_ai_controller.py` (line 34). New colors can be added by extending this dictionary:

```
COLOR_TABLE = {
    "red":    [1.0, 0.0, 0.0],
    "green":  [0.0, 1.0, 0.0],
    "blue":   [0.0, 0.0, 1.0],
    "yellow": [1.0, 1.0, 0.0],
    "white":  [1.0, 1.0, 1.0],
    "black":  [0.02, 0.02, 0.02],
    "purple": [0.8, 0.0, 0.8],
    "orange": [1.0, 0.5, 0.0],
    "cyan":   [0.0, 1.0, 1.0],
    "pink":   [1.0, 0.41, 0.71],
    "brown":  [0.55, 0.27, 0.07],
    "gray":   [0.5, 0.5, 0.5],
    "teal":   [0.0, 0.5, 0.5],
}
```

Hex codes (e.g., `#FF5733`) are converted using standard RGB division by 255. Unknown color names fall back to a default purple (`[0.8, 0.0, 0.8]`). The LLM prompt instructs it to use standard CSS color names or hex codes, and named colors are resolved server-side.

5.5 Action Schema and Supported Actions

Every user command is mapped to a JSON action object. The full set of supported top-level actions:

Table 5: Complete action schema.

Action	Parameters	Example
add_object	object_type, color, placement (optional)	<code>{"action": "add_object", "object_type": "sphere", "color": "blue"}</code>
move_object	target, direction or position	<code>{"action": "move_object", "target": "red_cube", "direction": "right"}</code>
delete_object	target	<code>{"action": "delete_object", "target": "sphere_1"}</code>
recolor_object	target, color	<code>{"action": "recolor_object", "target": "cube_1", "color": "green"}</code>
scale_object	target, factor or scale	<code>{"action": "scale_object", "target": "cube", "factor": 2.0}</code>
rotate_object	target, axis, degrees	<code>{"action": "rotate_object", "target": "cube", "axis": "y", "degrees": 45}</code>
apply_texture	target, texture_name	<code>{"action": "apply_texture", "target": "cube", "texture_name": "brick"}</code>
remove_texture	target	<code>{"action": "remove_texture", "target": "cube_1"}</code>
add_light	light_type, color (opt.), intensity (opt.), position/direction (opt.)	<code>{"action": "add_light", "light_type": "point"}</code>
delete_light	target	<code>{"action": "delete_light", "target": "light_1"}</code>
move_light	target, position	<code>{"action": "move_light", "target": "light_1", "position": [0, 8, 0]}</code>
change_light_color	target, color	<code>{"action": "change_light_color", "target": "light_1", "color": "orange"}</code>
change_light_intensity	target, intensity	<code>{"action": "change_light_intensity", "target": "light_1", "intensity": 0.5}</code>
add_prefab	prefab_name	<code>{"action": "add_prefab", "prefab_name": "tree"}</code>
generate_composite	composite, object_type, color (opt.)	<code>{"action": "generate_composite", "composite": "lamp", "object_type": "cylinder"}</code>
generate_pattern	pattern, object_type, count, radius, color (opt.)	<code>{"action": "generate_pattern", "pattern": "ring", "object_type": "cube", "count": 8}</code>
add_custom_model	model_path, color (opt.)	<code>{"action": "add_custom_model", "model_path": "chair.obj"}</code>
animate_object	target, animation_type, speed (opt.)	<code>{"action": "animate_object", "target": "cube_1", "animation_type": "bounce"}</code>
new_scene	(none)	<code>{"action": "new_scene"}</code>
save_scene	scene_name (optional)	<code>{"action": "save_scene", "scene_name": "my_tower"}</code>
load_scene	scene_name	<code>{"action": "load_scene", "scene_name": "my_tower"}</code>
undo	(none)	<code>{"action": "undo"}</code>
action_sequence	action_sequence (list)	<code>{"action": "action_sequence", "action_sequence": [...]}</code>

5.6 Feature Coverage

Table 6: Feature coverage. “Done” means at least 90% formal evaluation accuracy; “Partial” means limited support.

Feature / Action	Status	Notes
cube, sphere, cylinder, cone, pyramid, plane	Done	Phong shading
OBJ custom model	Done	No extra libs
USD / USDZ	Done	Requires usd-core
add_object	Done	shape, color, placement
delete / move / recolor_object	Done	
scale_object	Done	factor or [sx,sy,sz]
rotate_object	Done	x/y/z axis, any degrees
apply_texture / remove_texture	Done	8 textures
add_custom_model	Done	OBJ/USD via NL command
add/delete/move light	Partial	point and directional
generate_composite	Done	tree, table, lamp, etc.
add_prefab	Done	9 prefabs
action_sequence	Done	chains multiple actions
new / save / load scene	Done	named saves
undo	Done	file-based history stack
animate_object (bounce / spin / lerp)	Done	detected before LLM via regex
orbit object / light	Done	rule-based with <code>np.cos/sin</code> per frame
add_custom_model via NL	Done	bundled model keyword matching
create_group	Partial	rule-based only
redo	Not implemented	planned as future work

Figure 13 demonstrates all seven supported primitive types rendered in a single scene.

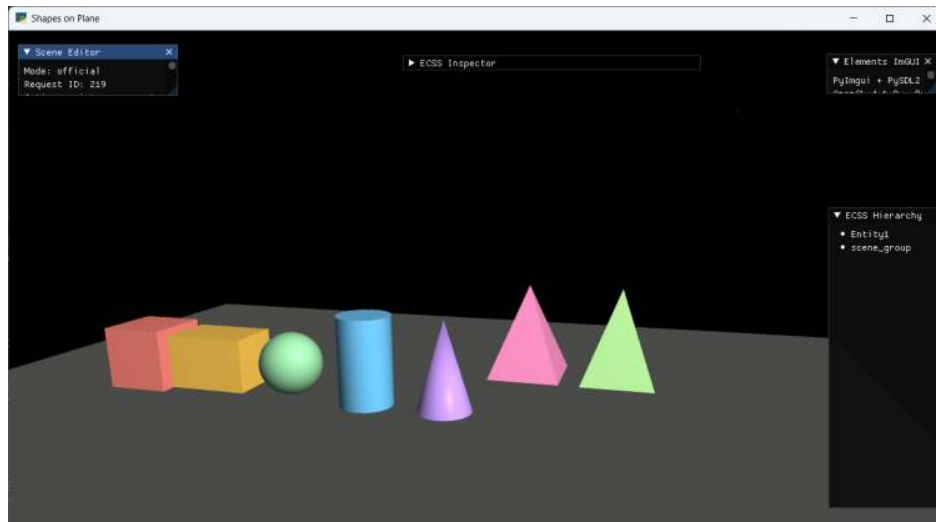


Figure 13: Scene demonstrating all seven supported primitive types (cube, rectangular prism, sphere, cylinder, cone, pyramid, triangular pyramid) and their ECSS hierarchy.

6 Evaluation

6.1 Automated Test Suite

The automated unit and integration test suite provides software correctness validation independent of LLM behavior [2]. It is located in `textToScene/tests/` and organized into focused test modules.

Table 7: Test modules and their coverage areas.

Test Module	Coverage Area
<code>test_code_generator.py</code>	Code generation correctness for all node types
<code>test_apply_action.py</code>	IR mutation correctness for all action types
<code>test_validate_action.py</code>	Action schema validation
<code>test_text_parser.py</code>	Rule-based parser
<code>test_prefabs.py</code> , <code>test_prefabs_detail.py</code>	Prefab construction and structure
<code>test_ir_helpers.py</code>	IR utility functions
<code>test_more_colors.py</code>	Color resolution
<code>test_move_up_down.py</code>	Directional movement geometry
<code>test_procedural.py</code>	Composite generation
<code>test_scale_object.py</code>	Scale action
<code>test_rotate_object.py</code>	Rotation action
<code>test_texture_actions.py</code>	Texture assignment and removal
<code>test_light_actions.py</code>	Light add/delete/move/color/intensity
<code>test_suite.py</code>	End-to-end scene construction integration tests
<code>test_animations_orbit.py</code>	Animation types (bounce/spin/lerp), orbit object/light, <code>detect_procedural_action</code>

The full suite contains 464 tests using pytest. Results:

Table 8: Automated test suite results.

Metric	Value
Total automated tests	464
Passing tests	464
Failing tests	0
Test pass rate	100%

The 102 tests added in `test_animations_orbit.py` cover the bounce, spin, and lerp

animation types, orbit detection for objects and lights, code generation correctness for animated objects, and the hybrid detection pipeline. All 464 tests pass.

Figure 14 shows the pass/fail breakdown by test module.

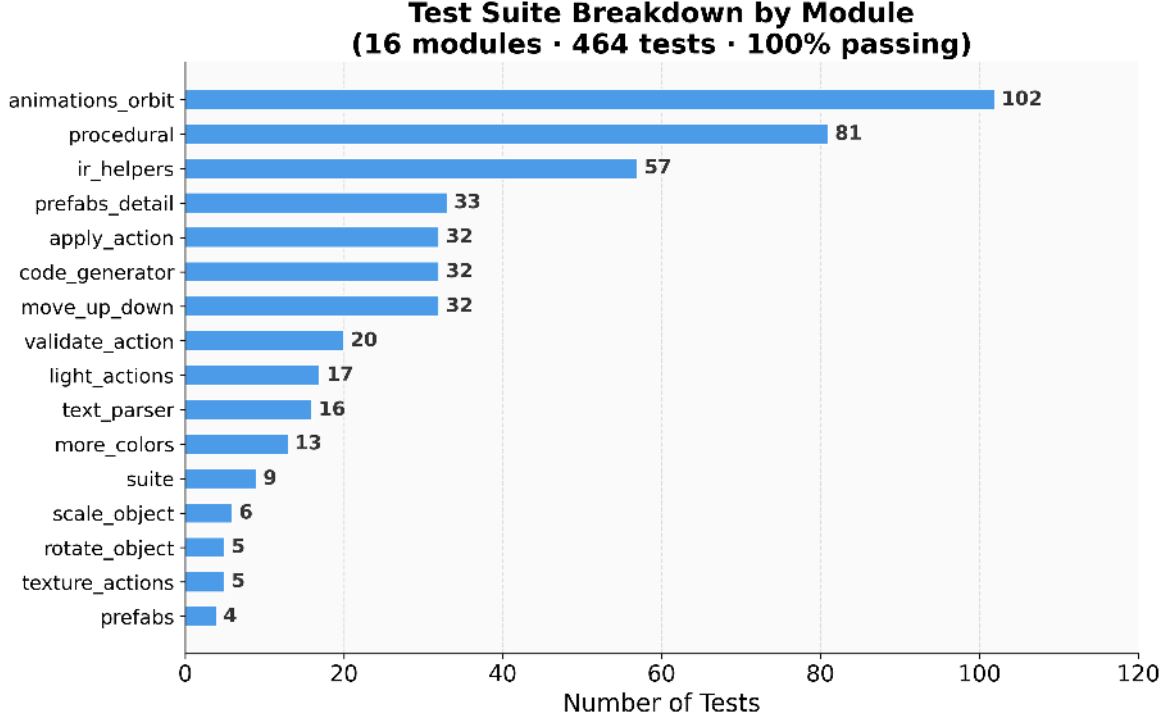


Figure 14: Automated test suite pass/fail breakdown by module. All 464 tests pass. The suite covers the core code-generation, action-application, animation, and orbit paths.

6.2 Experimental Setup

Test suite for LLM evaluation: The evaluation started with 125 natural language commands across 15 categories. All prompts were authored to reflect phrasing representative of beginner-level user commands. The same 125 prompts were submitted to all four LLM models and the rule-based baseline. The suite was subsequently extended by 21 prompts across four additional categories (Animation: 8, Orbit – Object: 5, Orbit – Light: 5, Custom Model Loading: 3), bringing the total to 146 commands across 19 categories. All five models (four LLM backends and the rule-based baseline) were evaluated on the full 146-command set.

Categories and command counts:

Category	Count
Object Creation	10
Movement Directional	15
Movement Positional	5
Recolor	10
Scale Natural Language	10
Scale Explicit	5
Rotation Simple	10
Rotation Axis	10
Deletion	5
Prefabs Add	8
Prefabs Transform	12
Composite Objects	10
Scene Management	5
Undo/Redo	5
Action Sequences	5
Animation	8
Orbit – Object	5
Orbit – Light	5
Custom Model Loading	3
Total (original)	125
Total (extended)	146

Evaluation mode: Cumulative scene mode: each command is evaluated in the context of the scene produced by all preceding commands in the same category block, reflecting realistic incremental usage. This choice was made to test the system under conditions that match its intended use (interactive scene building over time). An independent-mode evaluation (fresh scene per command) was not performed. This is acknowledged as a limitation.

Metrics (three binary metrics per command):

1. **Parse success:** LLM returned valid JSON with a recognised `action` field.
2. **Apply success:** action applied to scene IR without error.
3. **Intent accuracy** (primary): parse success AND apply success AND action type matches the expected type for that category.

All metrics are binary with no partial credit. Correctness is determined automatically with no manual annotation.

Models evaluated (pricing as of May 2026 at standard public API tier; rates vary and should be re-verified before reproducing cost estimates):

- GPT-4o-mini (OpenAI, \$0.15/1M input tokens at evaluation time)
- GPT-4.1-mini (OpenAI, \$0.40/1M at evaluation time)
- Gemini-2.5-Flash-Lite (Google, \$0.075/1M at evaluation time)
- Gemini-2.5-Flash (Google, \$0.15/1M at evaluation time)
- Rule-based fallback parser (no API cost, <3 ms)

Each model was run once at default temperature. Output variance is therefore not quantified.

Statistical analysis: For reference, treating each command as an independent Bernoulli trial (a simplification: commands run in cumulative mode, so successive commands share scene context and are not strictly independent), the 95% Wilson confidence interval for $n=125$ commands is given below. These intervals should be read as approximate bounds, not precise significance tests.

Table 10: Intent accuracy and 95% Wilson confidence intervals ($n = 125$).

Model	Accuracy	95% CI (Wilson)
Rule-Based	61.0%	[52.9%, 68.5%]
GPT-4o-mini	92.5%	[87.1%, 95.8%]
GPT-4.1-mini	90.4%	[84.5%, 94.2%]
Gemini-2.5-Flash-Lite	94.5%	[89.5%, 97.2%]
Gemini-2.5-Flash	89.7%	[83.7%, 93.7%]

6.3 Multi-Model Evaluation Results

Table 11 summarizes the overall results across all models.

Table 11: Overall results (146 commands, 19 categories, cumulative mode, single run per model). Cost figures use May 2026 public API pricing; actual costs may differ.

Metric	Rule-Based	GPT-4o-mini	GPT-4.1-mini	Gem-2.5-FL-Lite	Gem-2.5-Flash
Intent Accuracy	61.0%	92.5%	90.4%	94.5%	89.7%
Parse Success	97.9%	100%	100%	100%	98.6%
Apply Success	87.0%	97.9%	93.2%	97.9%	94.5%
Avg Latency (ms)	<3	4,120	4,113	1,583	5,730
Cost / 100 cmds	\$0.00	\$0.088	\$0.239	\$0.064	\$0.099
Tokens In	0	836,894	850,279	915,311	936,960
Tokens Out	0	5,245	5,244	5,104	5,921

Key findings:

- **Gemini-2.5-Flash-Lite achieves the highest intent accuracy at 94.5%** (138/146), outperforming GPT-4o-mini (92.5%), GPT-4.1-mini (90.4%), and Gemini-2.5-Flash (89.7%).
- **Parse success is 100% for three of the four LLM models.** Gemini-2.5-Flash achieves 98.6%, indicating two commands where the model returned unparseable output rather than a structured JSON action.
- **The Orbit categories are the weakest across all LLM models.** No model achieves better than 40% on Orbit – Object, and Gemini-2.5-Flash scores 0% on Orbit – Light, underscoring the value of the rule-based detection layer for these commands.
- **The rule-based baseline scores 61.0%**, establishing a lower bound and quantifying the gain from the LLM component. The 33.5 pp gap to Gemini-2.5-Flash-Lite narrows compared to the 15-category evaluation because the rule-based detector achieves 100% on Animation and Orbit.
- **In this evaluation, a smaller cheaper model (Gemini-2.5-Flash-Lite) outscored a larger one (Gemini-2.5-Flash)**, discussed further in Section 7.3.
- **Gemini-2.5-Flash-Lite is also the fastest** (1,583 ms average) and the cheapest LLM (\$0.064 per 100 commands), making it the most practical choice for interactive use in this evaluation.

Figure 15 displays these results.

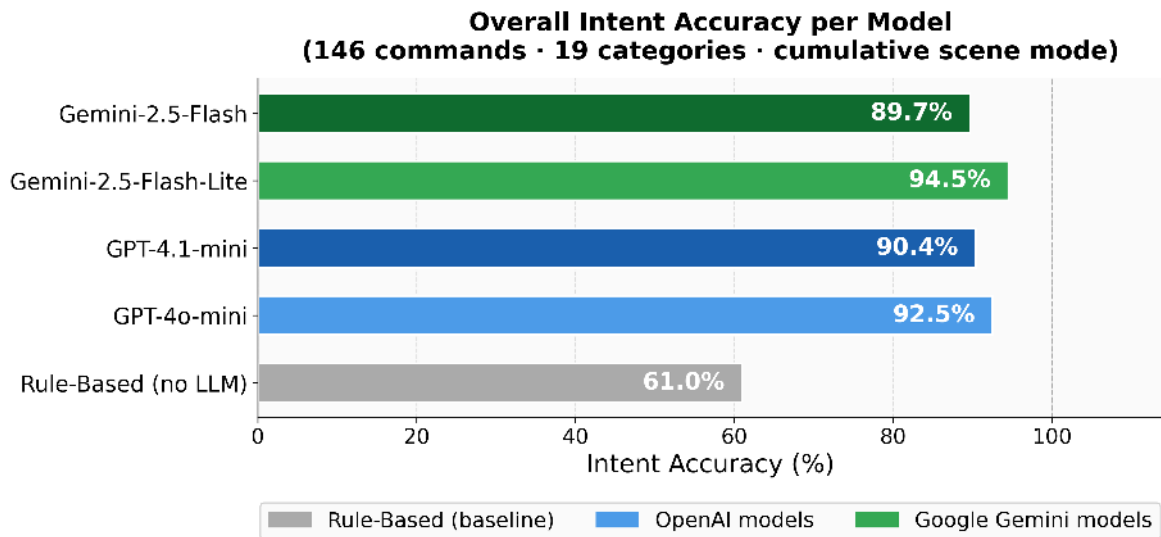


Figure 15: Overall intent accuracy across all evaluated models.

6.4 Per-Category Analysis

Table 12: Per-category intent accuracy (correct / total, percentage) across all 19 evaluation categories (146 commands total).

Category	GPT-4o-mini	GPT-4.1-mini	Gem-2.5-FL-Lite	Gem-2.5-Flash
Object Creation	9/10 (90%)	10/10 (100%)	10/10 (100%)	10/10 (100%)
Movement Directional	15/15 (100%)	15/15 (100%)	15/15 (100%)	15/15 (100%)
Movement Positional	5/5 (100%)	5/5 (100%)	5/5 (100%)	5/5 (100%)
Recolor	10/10 (100%)	10/10 (100%)	10/10 (100%)	10/10 (100%)
Scale – Nat. Language	10/10 (100%)	10/10 (100%)	10/10 (100%)	10/10 (100%)
Scale – Explicit	5/5 (100%)	5/5 (100%)	5/5 (100%)	5/5 (100%)
Rotation – Simple	10/10 (100%)	10/10 (100%)	10/10 (100%)	10/10 (100%)
Rotation – Axis	10/10 (100%)	10/10 (100%)	10/10 (100%)	10/10 (100%)
Deletion	5/5 (100%)	5/5 (100%)	5/5 (100%)	5/5 (100%)
Prefabs Add	8/8 (100%)	8/8 (100%)	8/8 (100%)	8/8 (100%)
Prefabs Transform	12/12 (100%)	6/12 (50%)	10/12 (83%)	9/12 (75%)
Composite Objects	9/10 (90%)	10/10 (100%)	10/10 (100%)	10/10 (100%)
Animation	4/8 (50%)	8/8 (100%)	7/8 (88%)	5/8 (62%)
Orbit – Object	0/5 (0%)	0/5 (0%)	2/5 (40%)	1/5 (20%)
Orbit – Light	5/5 (100%)	3/5 (60%)	3/5 (60%)	0/5 (0%)
Custom Model Loading	3/3 (100%)	3/3 (100%)	3/3 (100%)	3/3 (100%)
Scene Management	5/5 (100%)	5/5 (100%)	5/5 (100%)	5/5 (100%)
Undo/Redo	5/5 (100%)	4/5 (80%)	5/5 (100%)	5/5 (100%)
Action Sequences	5/5 (100%)	5/5 (100%)	5/5 (100%)	5/5 (100%)

Analysis of key categories:

Object Creation (GPT-4o-mini: 90%): The single failure on GPT-4o-mini was due to ambiguous spatial placement inference within the cumulative scene context, where a directional relation was resolved against the wrong target object.

Prefabs Transform: This is the category with the largest inter-model variance. GPT-4o-mini achieves 100% (12/12), while GPT-4.1-mini scores only 50% (6/12), Gemini-2.5-Flash-Lite reaches 83% (10/12), and Gemini-2.5-Flash achieves 75% (9/12). Prompts in this category ask to transform (move, scale, rotate) an existing prefab group. Failures occur when the model targets a child part rather than the top-level group name, or generates an `action_sequence` where a single targeted transform was

expected.

Animation: GPT-4.1-mini achieves 100% (8/8), Gemini-2.5-Flash-Lite 88% (7/8), Gemini-2.5-Flash 62% (5/8), and GPT-4o-mini 50% (4/8). Failures arise when the model returns an `add_object` action rather than `animate_object`. The rule-based detector handles all animation commands reliably by keyword matching before the LLM is consulted.

Orbit – Object and Orbit – Light: These are the weakest categories across all LLM models. On Orbit – Object, GPT-4o-mini and GPT-4.1-mini both score 0% (0/5), Gemini-2.5-Flash-Lite reaches 40% (2/5), and Gemini-2.5-Flash achieves 20% (1/5). On Orbit – Light, GPT-4o-mini achieves 100% (5/5), GPT-4.1-mini and Gemini-2.5-Flash-Lite both score 60% (3/5), and Gemini-2.5-Flash scores 0% (0/5). The dominant failure mode is the model returning a plain `add_object` or `add_light` action without the orbit field, demonstrating that orbit motion is an unconventional enough concept that LLM-only approaches struggle with it. The rule-based detector achieves 100% on both orbit categories, as discussed in Section 6.6.

Custom Model Loading achieves 100% across all four LLM models (3/3 each), confirming that the `add_custom_model` action is well-specified and consistently recognised from commands such as “load a chameleon” or “add the teapot”.

Perfect categories (100% for all LLM models): Movement (Directional and Positional), Recolor, Scale (Natural Language and Explicit), Rotation (Simple and Axis), Deletion, Prefabs Add, Custom Model Loading, Scene Management, and Action Sequences. These are well-defined, atomic operations with unambiguous natural language phrasing that all four models handle without error.

Per-Category Intent Accuracy (LLM models only, 4 backends · 19 categories)

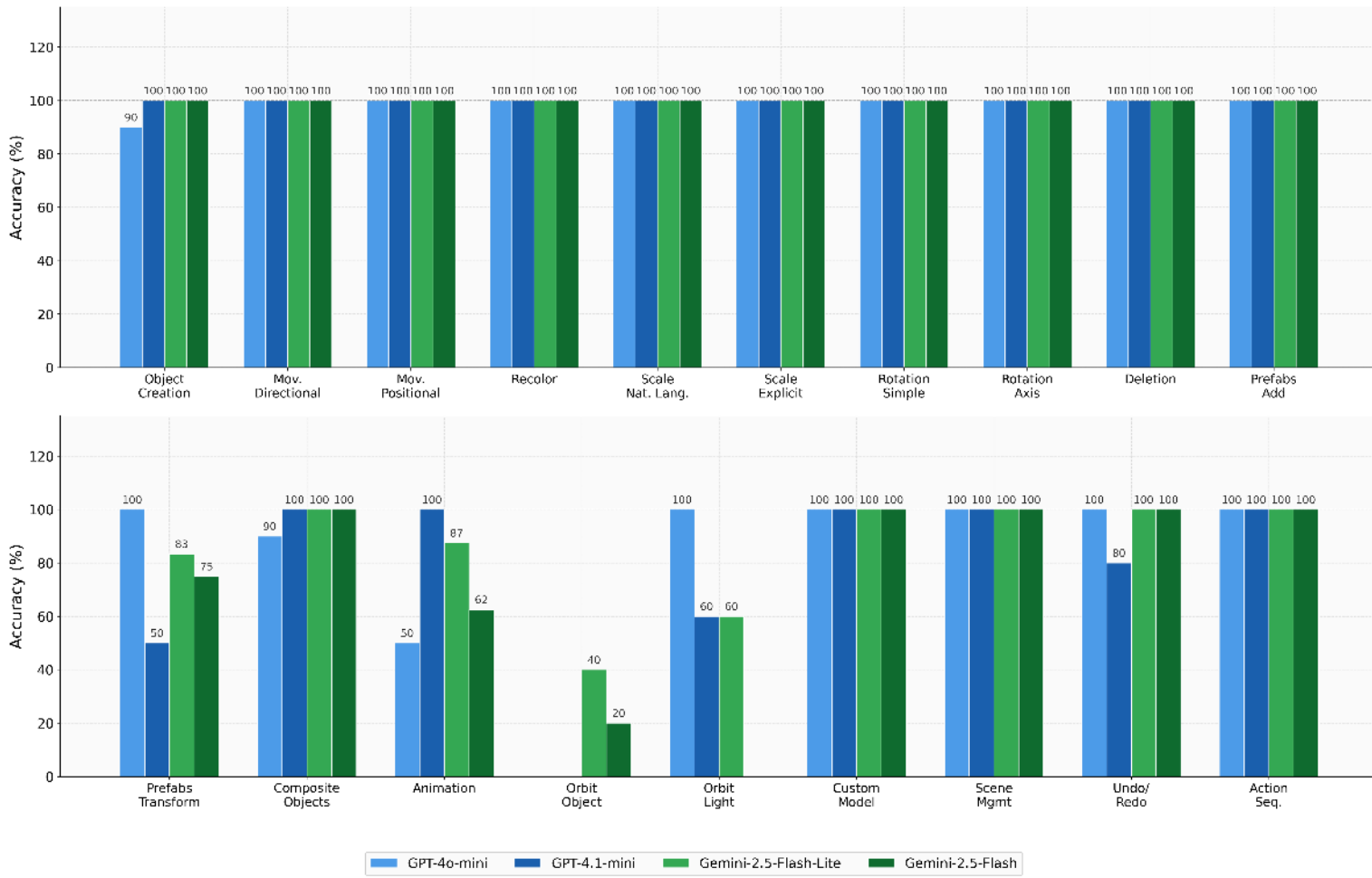


Figure 16: Per-category intent accuracy (LLM models only; rule-based baseline excluded for clarity).

Figure 16 shows the full per-category accuracy breakdown for all LLM models.

6.5 Latency and Cost Analysis

Latency: Gemini-2.5-Flash-Lite averages 1,583 ms per command (roughly 1.6 seconds from prompt submission to preview ready), making it the fastest LLM option by a large margin. GPT-4o-mini and GPT-4.1-mini average approximately 4.1 seconds each (4,120 ms and 4,113 ms respectively). Gemini-2.5-Flash is the slowest at 5,730 ms, nearly four times slower than Gemini-2.5-Flash-Lite. For interactive use, sub-two-second latency remains within acceptable responsiveness thresholds. At nearly six seconds, Gemini-2.5-Flash is likely to disrupt interactive flow.

Cost: The cost ranking (cheapest first) is: Rule-based (\$0.00), Gemini-2.5-Flash-Lite (\$0.064/100), GPT-4o-mini (\$0.088/100), Gemini-2.5-Flash (\$0.099/100), GPT-4.1-mini

(\$0.239/100). GPT-4.1-mini is approximately $3.7\times$ more expensive than Gemini-2.5-Flash-Lite while delivering lower intent accuracy (90.4% vs 94.5%).

As a rough illustration based on June 2026 pricing, a classroom deployment with 30 students each submitting 50 commands per session would cost approximately: $30 \times 50 \times \$0.00064 \approx \mathbf{\$0.96 \text{ per session}}$ using Gemini-2.5-Flash-Lite. This figure is indicative only; actual costs depend on current provider pricing, tier, and caching behaviour.

The action cache further reduces costs. In a typical beginner session, many commands are standard (“add a red cube”, “move it right”) and will hit the cache after the first use by any student. In practice, a warm cache would reduce API calls for repeated commands.

Figure 17 plots latency against cost for all evaluated models.

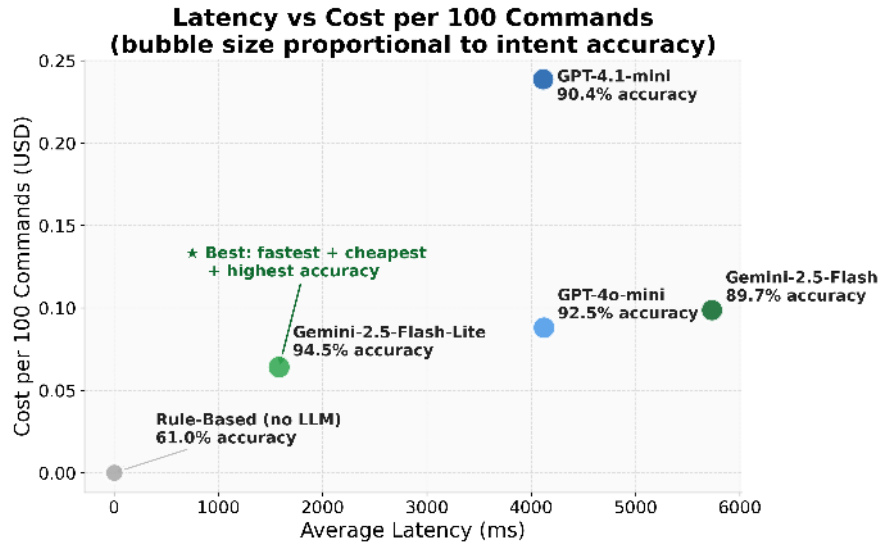


Figure 17: Latency (ms) vs. cost per 100 commands for all evaluated models (May 2026 pricing).

6.6 Rule-Based Baseline Reference

The same 146 commands were executed through the built-in rule-based fallback parser (regex and keyword matching, no API call, <3 ms):

Table 13: Rule-based baseline vs. `textToScene` (Gemini-2.5-Flash-Lite) across all 19 categories. Bold values indicate the higher score per category.

Object Creation	100%	100%	0 pp
Movement Directional	100%	100%	0 pp
Movement Positional	100%	100%	0 pp
Recolor	10%	100%	+90 pp

Category	Rule-Based	Gem-2.5-FL-Lite	Delta Gain
Scale - Nat. Language	90%	100%	+10 pp
Scale - Explicit	100%	100%	0 pp
Rotation - Simple	0%	100%	+100 pp
Rotation - Axis	0%	100%	+100 pp
Deletion	100%	100%	0 pp
Prefabs - Add	38%	100%	+63 pp
Prefabs - Transform	42%	100%	+58 pp
Composite Objects	20%	100%	+80 pp
Scene Management	100%	100%	0 pp
Undo/Redo	80%	80%	0 pp
Action Sequences	0%	80%	+80 pp
Animation	100%	88%	-12 pp
Orbit - Object	100%	40%	-60 pp
Orbit - Light	100%	60%	-40 pp
Custom Model Loading	100%	100%	0 pp
Overall (orig. 15 cat., $n = 125$)	55.2%	99.2%	+44 pp
Overall (all 19 cat., $n = 146$)	61.0%	94.5%	+33.5 pp

Figure 18 visualises the per-category gap between the rule-based baseline and Gemini-2.5-Flash-Lite.

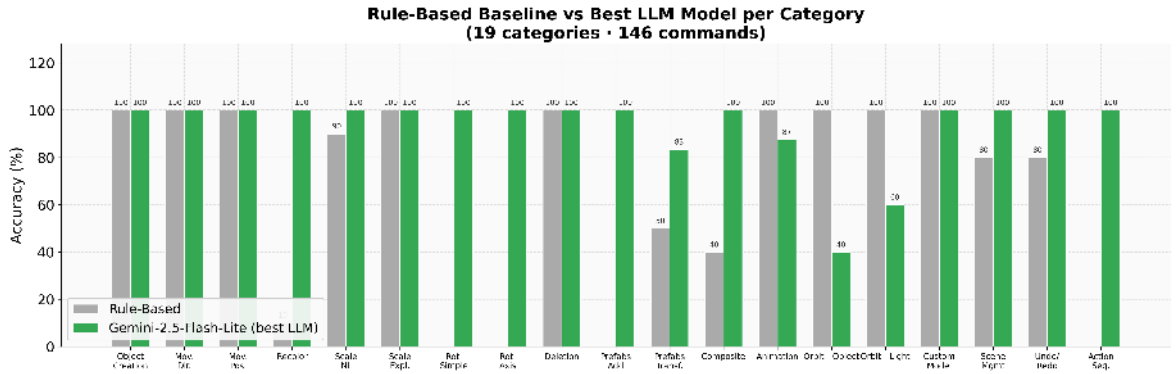


Figure 18: Per-category comparison of the rule-based baseline against Gemini-2.5-Flash-Lite.

The rule-based parser works well for rigid, predictable commands: “add cube”, “move right”, “delete”, and basic scene management. It completely fails for rotation (no angle parsing implemented), indirect recolor phrasing (“paint the cube red” scores only 10%), composite generation, and action sequences. The 44 pp gain on the original 15 categories quantifies the value added by natural language understanding over keyword matching.

A notable reversal appears in the four new categories. For Animation and Orbit, the rule-based detector achieves 100% while Gemini-2.5-Flash-Lite scores only 40–88%. This is explained by the hybrid pipeline design: these commands are intercepted by `detect_procedural_action` before the LLM is invoked, so the rule-based component benefits from vocabulary it was explicitly designed for. The LLM, in contrast, tends to return a plain `add_object` action without the orbit field, missing the intent. This finding supports the hybrid pipeline as a design rationale: routing structurally deterministic intents through regex detection before the LLM reduces both latency and error rate for those categories.

Figure 19 shows the per-model accuracy on the Animation, Orbit, and Custom Model Loading categories across all five evaluated models.

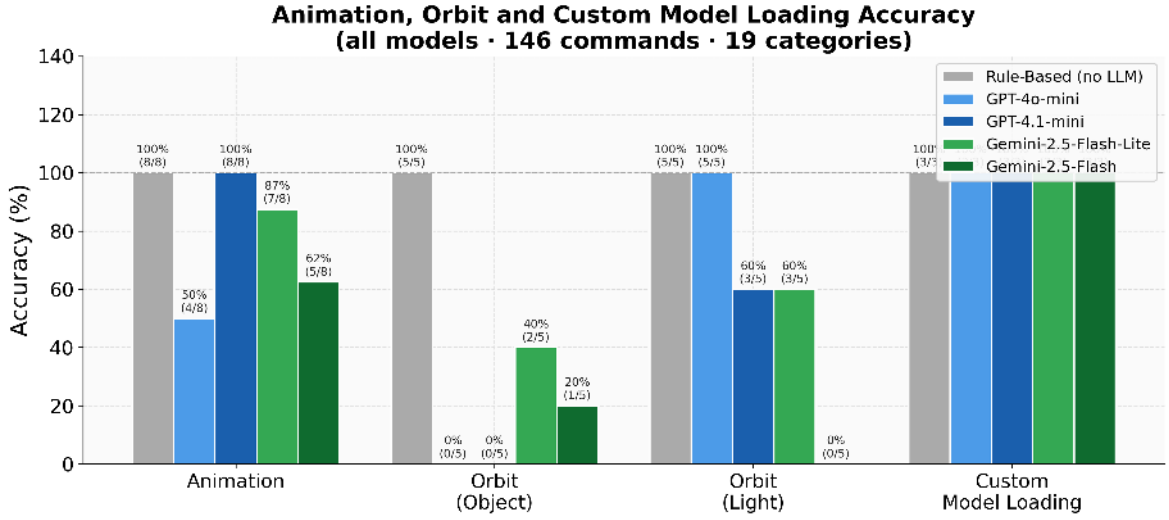


Figure 19: Accuracy on the Animation, Orbit (Object and Light), and Custom Model Loading categories for all evaluated models. The rule-based detector achieves 100% on all motion-related categories, while LLM models score between 0% and 100% depending on category, illustrating the advantage of the hybrid detection pipeline.

6.7 Few-Shot Direct Code Generation Baseline

A direct code generation baseline was evaluated to quantify the gap between few-shot LLM code generation and the `textToScene` structured pipeline. The baseline provides three complete Elements scene scripts as in-context examples and asks the model to generate a new script from a natural language prompt, with no intermediate JSON action or IR layer.

The evaluation used Gemini-2.5-Flash-Lite on 20 prompts covering single-object scenes, multi-object compositions, and objects with explicit scale or position. Each generated script was assessed on three binary criteria: syntax validity (Python compiles without error), API completeness (all required Elements patterns present, including

`scene.init()`, `scene.render()`, `VertexArray()`, `ShaderGLDecorator`), and absence of hallucinated API calls.

Table 14: Few-shot direct code generation vs. `textToScene` (Gemini-2.5-Flash-Lite, 20 prompts).

Metric	Few-Shot LLM	<code>textToScene</code>
Syntax valid	18/20 (90%)	20/20 (100%)
API complete	0/20 (0%)	20/20 (100%)
No hallucinations	20/20 (100%)	20/20 (100%)
Full pass	0/20 (0%)	20/20 (100%)

The few-shot baseline achieved 0% full pass. The consistent failure was omission of render loop initialization calls (`scene.init()` and `scene.render()`), which appear in the example scripts but were not reproduced reliably across prompts. Syntax was largely valid (90%) and no hallucinated API methods were observed, meaning the model does learn the import patterns from the examples. Complete render loop structure was not reliably transferred from three examples.

`textToScene`’s deterministic code generator achieves 100% on all three metrics by construction, since it generates from a validated IR using fixed code templates that always include the full render loop.

6.8 Error Analysis

Failure mode 1: Wrong action type. The model returns a syntactically valid action but of the wrong type. Example: user says “put the lamp on the table”: model returns `move_object` (correct) but targeting “table” instead of “lamp” (wrong). This accounts for the majority of failures in Object Creation for GPT-4o-mini.

Failure mode 2: Missing required fields. Rare with the current prompt (0 cases in the formal evaluation), but observed in informal testing when the prompt is very terse. Example: a very short rotation command might cause the model to omit the `axis` field.

Failure mode 3: Action sequence fragmentation. The model generates the first action of a sequence but forgets the subsequent ones. Example: given “add a blue cube and make it twice as big”, the model returns only `add_object`, omitting the scale step.

Failure mode 4: Group/prefab name confusion. When the cumulative scene contains both a prefab group (e.g., “tree_1”) and its children (e.g., “tree_1_trunk”), commands targeting “the tree” may be applied to a child part rather than the group, causing incorrect scaling or movement.

Failure mode 5: Placement inference. Spatial relations (“behind”, “to the left of”, “in front of”) require 3D spatial reasoning. The LLM sometimes uses the wrong direction vector for the relation, particularly when the scene camera orientation differs from the user’s mental frame of reference.

7 Discussion

7.1 Explainability and Transparency

A key advantage of the structured pipeline over direct few-shot code generation is explainability. In a few-shot approach the model’s intent is hidden, errors appear as Python runtime exceptions, and no intermediate state is available for inspection or approval. The `textToScene` pipeline provides five explainability mechanisms:

1. Every NL command maps to a human-readable JSON action object, visible in `ai_request.json`.
2. The scene is maintained as a structured IR (`scene_ir.json`) independent of generated code.
3. The preview-apply workflow requires explicit user approval.
4. The code generator is deterministic: the same IR always produces the same Python output.
5. The undo history stack allows any change to be reversed. Every past state is visible as a JSON snapshot in `history/`.

These properties align with explainable AI (XAI) and human-in-the-loop design principles [1].

Table 15: Explainability: direct few-shot generation vs. `textToScene`.

Property	Direct Few-Shot LLM	<code>textToScene</code> (ours)
Intermediate representation	None	Human-readable JSON IR
Action transparency	Hidden in model	Explicit JSON action object
Code validity guarantee	No (hallucinations possible)	Structural check only (deterministic from IR; intent errors still possible)
User approval step	No	Preview-apply workflow
Undo support	No	Full history stack
Debuggability	Stack trace only	Pinpointed to pipeline stage

7.2 Educational Value

Whether the system has measurable educational value has not been tested in a formal user study (acknowledged as a limitation in Section 7.4). The following design choices

are intended to support learning, but this intention is not empirically validated.

Visible code: Every action produces a new, complete Python script that the student can open, read, modify, and run independently. The generated code is verbose by design, each ECS call is on its own line, each uniform is named explicitly with the intent that this verbosity makes the mapping from scene to code readable.

Visible hierarchy: The IR JSON exposes the parent-child organization of the scene. For example, a house entity shows its body and roof children, making hierarchical transforms visible.

Visible intermediate steps: The JSON action object shows what the LLM interpreted from the student’s prompt. If the action is wrong (e.g., the model returned `scale: 2.0` but the student meant to rotate), the student sees this before the action is applied and can reject it and rephrase a capability not available in direct code synthesis.

7.3 Model Selection Insights

In this evaluation, Gemini-2.5-Flash-Lite (a smaller, cheaper model) scored higher than Gemini-2.5-Flash (a larger, more expensive model). The most plausible explanation is over-interpretation: Gemini-2.5-Flash failed 67% of Prefabs Transform commands by generating an `action_sequence` where a single targeted action was expected. The task requires exactly one JSON object from a finite schema, and larger models appear to produce more elaborate responses that do not match the expected structure. This is based on four models in one evaluation run, so the pattern should not be generalized without a broader comparison.

7.4 Limitations

1. **Single-run evaluation:** Temperature-driven randomness is not accounted for.
2. **Cumulative evaluation mode:** Context carry-over between commands within a category block may inflate or deflate results compared to independent evaluation. Future work should include independent-mode runs.
3. **Scene complexity:** The system is tested on beginner-level scenes (simple primitives, basic prefabs). Complex scenes with many objects may challenge the LLM’s context resolution.
4. **No redo:** The undo stack supports arbitrary depth. Redo is not implemented.
5. **Single active light:** The IR can store multiple light nodes, but only the first is used for rendering.
6. **Group creation via NL:** The `create_group` action is implemented in the

rule-based parser but not connected to the main LLM pipeline.

7. **No voice input:** The current interface is text-only via the ImGui panel.
8. **No PBR materials:** The system uses Phong shading. Physically-based rendering (PBR) materials are not supported.
9. **Single-model few-shot evaluation:** The direct code generation baseline in Section 6.7 was run with Gemini-2.5-Flash-Lite only and on 20 prompts. Additional models and a larger prompt set were not evaluated.
10. **No user study:** The educational value claims in Section 7.2 are based on design intent. No formal evaluation with students was conducted.

8 Conclusion and Future Work

8.1 Conclusion

This thesis described `textToScene`, a structured NL-to-code pipeline for the Elements graphics framework, and evaluated it across up to 146 commands, 19 categories, four LLM backends, and a rule-based baseline.

The main empirical finding is that Gemini-2.5-Flash-Lite achieves 94.5% intent accuracy on the full 146-command set (99.2% on the original 125-command benchmark) at the lowest cost (\$0.064 per 100 commands) among all evaluated models. An additional finding is that the hybrid pipeline design – routing animation and orbit commands through rule-based detection before the LLM – outperforms the LLM on those categories (100% vs 40–88%), supporting the design rationale for `detect_procedural_action`. The automated software test suite (464 tests, 100% pass rate) validates implementation correctness independently.

The design contribution is the structured IR layer: by interposing a validated JSON action object between LLM output and code generation, the system avoids the non-determinism and unauditable behavior of direct code synthesis. Whether the pipeline’s transparency properties translate into measurable learning gains remains an open empirical question, as no user study was conducted.

8.2 Future Work

Near-term improvements. Several concrete improvements can be addressed with modest engineering effort. Redo support would extend the existing file-based history stack to support forward replay in addition to backward undo. The `create_group` action is already implemented at the IR level but not yet wired into the LLM pipeline; connecting it would allow users to name and organize scene hierarchies through natural language. The Phong shader currently supports a single light source; adding multi-light support would enable richer scene illumination without changes to the action schema. Finally, integrating a speech-to-text API such as Whisper would make hands-free scene editing possible, widening accessibility.

Medium-term extensions. At the medium term, the most impactful improvement would be a semantic placement module to replace the current geometric heuristics. The present implementation computes “on top of” as a bounding-box y-offset, which is insufficient for complex spatial relations such as “between”, “in a circle around”, or “diagonally from”; a dedicated spatial reasoning module would improve accuracy in these cases. On the rendering side, extending the shader to support metallic/roughness

PBR materials would increase visual fidelity, and preserving material properties (albedo, roughness, metallic) from imported USD stages (rather than applying a flat color override) would make USD import more faithful. An export path to USD would additionally allow generated scenes to be opened in Omniverse, Blender, or other DCC tools.

Long-term research directions. The longer-term vision for `textToScene` is as a component in an educational platform for computer graphics. Embedding the system in an interactive tutorial environment would allow tutorial steps to issue specific NL commands, with the system automatically validating that the resulting scene matches expected outcomes. Learner modeling (i.e., tracking which concepts, such as transformations, materials, and scene hierarchy, a student has already exercised) would allow the platform to adapt scene suggestions to individual progress. Multi-turn dialogue, where the system asks clarifying questions such as “Did you mean to scale only the height, or all dimensions?”, would make the interaction more robust to ambiguous commands. A formal user study with introductory graphics students remains the most important open question: it would determine whether the system genuinely improves conceptual understanding compared to a traditional code-first approach. Finally, extending code generation to target other educational frameworks (Blender Python API, Three.js, OpenGL with GLFW) would broaden the potential student audience beyond the Elements framework.

References

- [1] A. Adadi and M. Berrada. Peeking inside the black-box: A survey on explainable artificial intelligence (XAI). *IEEE Access*, 6:52138–52160, 2018.
- [2] S. Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, B. Nushi, and T. Zimmermann. Software engineering for machine learning: A case study. In *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice*, pages 291–300. IEEE Press, 2019.
- [3] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. V. Le, and C. Sutton. Program synthesis with large language models, 2021.
- [4] J. Betker, G. Goh, L. Jing, T. Brooks, J. Wang, L. Li, L. Ouyang, J. Zhuang, J. Lee, Y. Guo, et al. Improving image generation with better captions. Technical report, OpenAI, 2023.
- [5] Blender Foundation. Blender – A 3D modelling and rendering package. <https://www.blender.org>, 2024.
- [6] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, and D. Amodei. Language models are few-shot learners, 2020.
- [7] M. J. J. Bucher and I. Armeni. ReSpace: Text-driven 3D scene synthesis and editing with preference alignment, 2025.
- [8] A. Chang, M. Savva, and C. D. Manning. Learning spatial knowledge for text to 3D scene generation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2028–2038, Doha, Qatar, 2014. Association for Computational Linguistics.
- [9] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. Ponde, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, and W. Zaremba. Evaluating large language models trained on code, 2021.
- [10] A. Haque, M. Tancik, A. A. Efros, A. Holynski, and A. Kanazawa. Instruct-NeRF2NeRF: Editing 3D scenes with instructions. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 19740–19750, 2023.

- [11] Z. Hu, A. Iscen, A. Jain, T. Kipf, Y. Yue, D. A. Ross, C. Schmid, and A. Fathi. SceneCraft: An LLM agent for synthesizing 3D scenes as blender code. In *Forty-First International Conference on Machine Learning*, 2024.
- [12] H. Jun and A. Nichol. Shap-e: Generating conditional 3D implicit functions, 2023.
- [13] C.-H. Lin, J. Gao, L. Tang, T. Takikawa, X. Zeng, X. Huang, K. Kreis, S. Fidler, M.-Y. Liu, and T.-Y. Lin. Magic3D: High-resolution text-to-3D content creation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 300–309, 2023.
- [14] M. Liu, C. Xu, H. Jin, L. Chen, M. Varma T, Z. Xu, and H. Su. One-2-3-45: Any single image to 3D mesh in 45 seconds without per-shape optimization. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [15] A. Nichol, H. Jun, P. Dhariwal, P. Mishkin, and M. Chen. Point-e: A system for generating 3D point clouds from complex prompts, 2022.
- [16] B. M. Öcal, M. Tatarchenko, S. Karaoglu, and T. Gevers. SceneTeller: Language-to-3D scene generation. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 358–375. Springer, 2024.
- [17] OpenUSD Alliance. Universal Scene Description. <https://openusd.org>, 2023.
- [18] G. Papagiannakis, M. Kamarianakis, A. Protopsaltis, D. Angelis, and P. Zikas. Project elements: A computational entity-component-system in a scene-graph pythonic framework, for a neural, geometric computer graphics curriculum. In *Eurographics 2023 – Education Papers*. The Eurographics Association, 2023.
- [19] B. Poole, A. Jain, J. T. Barron, and B. Mildenhall. DreamFusion: Text-to-3D using 2D diffusion, 2022.
- [20] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10684–10695, 2022.
- [21] Unity Technologies. Unity real-time development platform. <https://unity.com>, 2024.
- [22] K. VanLehn. The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. *Educational Psychologist*, 46(4):197–221, 2011.

- [23] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.
- [24] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.

Appendix

Appendix A: Example Workflow Trace

Complete trace of the command “Add a blue sphere on top of the red cube”:

Step 1: User types in the ImGui panel.

Step 2: Cache miss; LLM call made (Gemini-2.5-Flash-Lite). Scene context passed:

```
{
  "objects": [
    { "name": "cube_1", "id": "cube_1", "shape": "cube",
      "color": [0.8, 0.0, 0.0], "position": [0.0, 0.5, 0.0] }
  ]
}
```

Step 3: LLM returns JSON action:

```
{
  "action": "add_object",
  "object_type": "sphere",
  "color": "blue",
  "placement": {
    "relation": "on_top_of",
    "target": "red cube"
  }
}
```

Step 4: `apply_action_to_ir()` resolves `target: "red cube"` to `cube_1`, computes placement position (top of cube bounding box: $y = 0.5 + 0.5 + 0.5 = 1.5$), creates new IR node:

```
{
  "node_type": "mesh_object",
  "name": "sphere_1",
  "id": "sphere_1",
  "shape": "sphere",
  "transform": { "position": [0.0, 1.5, 0.0], "scale": [1.0, 1.0, 1.0] },
  "material": { "color": [0.0, 0.0, 1.0], "texture": {"enabled": false} }
}
```

Step 5: `generate_scene_script()` emits:

```
# ===== mesh_object: sphere_1 =====
vertices_2 = np.array([[...], ...], dtype=np.float32) # sphere
vertices
```

```

indices_2 = np.array([[...], ...], dtype=np.uint32)
colors_2 = np.array([[0.0, 0.0, 1.0, 1.0], ...], dtype=np.float32)
normals_2 = np.array([[...], ...], dtype=np.float32)

node_2 = scene.world.createEntity(Entity(name="sphere_1"))
scene.world.addEntityChild(rootEntity, node_2)
trans_2 = scene.world.addComponent(node_2,
    BasicTransform(name="sphere_1_TRS",
        trs=util.translate(0.0, 1.5, 0.0) @ util.identity
        ()))
mesh_2 = scene.world.addComponent(node_2, RenderMesh(name="
    sphere_1_mesh"))
mesh_2.vertex_attributes.append(vertices_2)
mesh_2.vertex_attributes.append(colors_2)
mesh_2.vertex_attributes.append(normals_2)
mesh_2.vertex_index.append(indices_2)
scene.world.addComponent(node_2, VertexArray())
shader_2 = scene.world.addComponent(node_2, ShaderGLDecorator(
    Shader(vertex_source=Shader.VERT_PHONG_MVP,
        fragment_source=Shader.FRAG_PHONG)))

```

Step 6: Preview script hot-swapped in. The user sees the blue sphere sitting on top of the red cube before committing the change. Figure 20 shows this preview state.

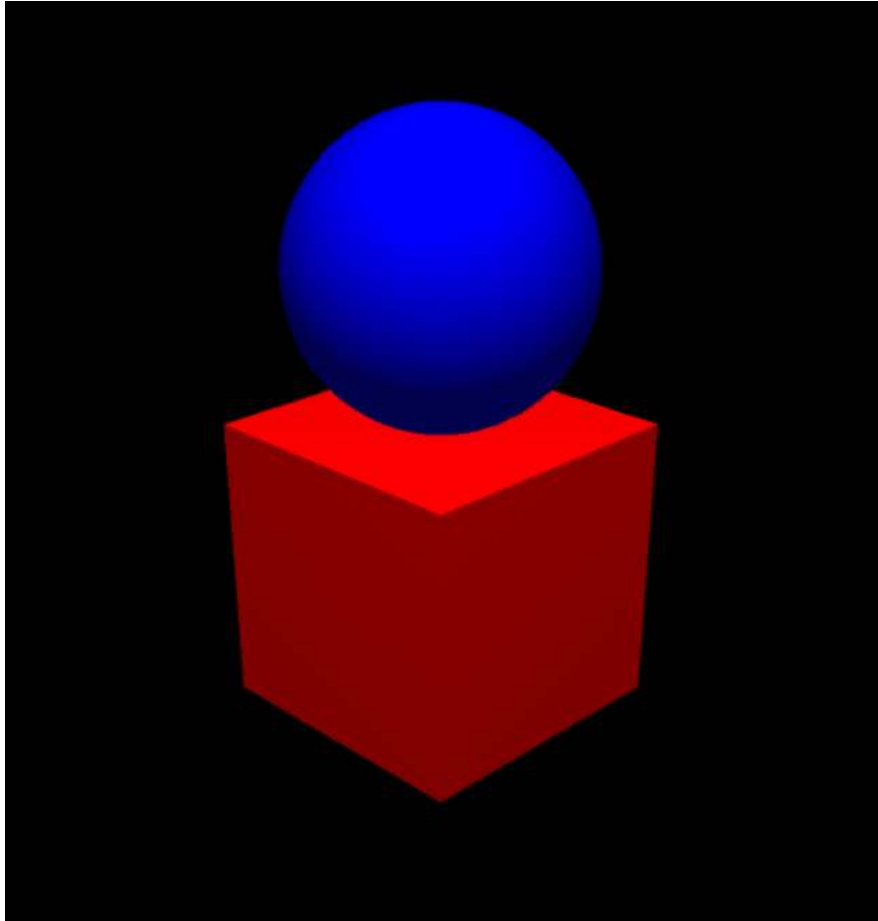


Figure 20: Preview of “Add a blue sphere on top of the red cube” before the user clicks Apply.

Step 7: User clicks “Apply”. Canonical IR updated. Undo snapshot saved. Official `scene_out.py` regenerated.

Appendix B: Sample LLM Prompt (Abridged)

The system prompt sent to the LLM is approximately 6,300 tokens. A structural overview:

```
You are a strict scene-editing parser.  
Return exactly one JSON object.  
[... format constraints ...]
```

```
Allowed top-level action values:
```

```
- add_object  
- move_object  
[... 22 actions total ...]
```

```
Rules:
```

- For rotate_object: use { "action":"rotate_object", "target":"...", "axis":"x|y|z", "degrees":<number> }
- For scale_object: "make it bigger" \textrightarrow{} factor 1.5, "double" \textrightarrow{} factor 2.0, ...
- For apply_texture, texture_name must be one of: brick, wood, stone, grass, metal, sand, marble, concrete

[... additional rules ...]

Examples:

```
{ "action": "add_object", "object_type": "cube", "color": "red" }
{ "action": "move_object", "target": "blue cube", "direction": "right" }
[... ~30 examples total ...]
```

User prompt:

Add a blue sphere on top of the red cube

Scene context:

```
{ "objects": [ { "name": "cube_1", "shape": "cube", "color": [1,0,0], "position": [0,0.5,0] } ] }
```

Appendix C: Folder Structure

```
~/.textToScene/scene_bridge/
|----- scene_ir.json           # Canonical scene IR
|----- preview_scene_ir.json   # Staged preview IR
|----- ai_request.json         # Current request (status +
    parsed action)
|----- ui_state.json           # User UI actions (apply/reject/
    undo)
|----- scene_state.json        # Active script and mode
|----- preview_scene.py        # Auto-generated preview Python
    script
|----- cache/
|   '----- action_cache.json   # Prompt → action cache
|----- custom_models/
|   '----- *.obj / *.usd       # User-supplied 3D assets
|----- textures/
|   '----- brick.jpg, wood.jpg... # Texture image assets
|----- history/
|   '----- undo_stack.json     # File-based undo history
'----- saved_scenes/
    '----- my_scene.json       # User-named saved scenes

~/.textToScene/
'----- scene_out.py            # Official (applied) Python scene
    script
```

```

textToScene/src/
|----- config.py                # Path constants, API keys, model
    selection
|----- llm_parser.py            # LLM communication, prompt
    building, cache
|----- mock_ai_controller.py    # Command dispatch, IR mutation,
    undo
|----- code_generator.py        # Deterministic IR \
    textrightarrow{} Python translation
|----- geometry_factory.py      # Parametric mesh generation
|----- prefabs.py              # Prefab definitions (house, tree
    , etc.)
|----- ir_schema.py            # IR schema validation
'----- supervisor.py           # Process management, hot-swap

textToScene/tests/
|----- test_code_generator.py
|----- test_apply_action.py
|----- test_validate_action.py
[... 16 test modules, 464 tests total ...]
'----- evaluation_runner.py     # Multi-model LLM benchmark
    runner

```

End of Report

Elisavet Kourkounaki

Student ID: CSD5116

Department of Computer Science, University of Crete